KOCAELI UNIVERSITY
INSTITUTE OF SCIENCE

ELECTRONICS AND COMMUNICATIONS ENGINEERING DEPARTMENT

MASTER'S THESIS

APPLICATION OF AN OBJECT DETECTION-BASED PLC-CONTROLLED SORTING SYSTEM USING DELTA ROBOT

MOHAMMED ELKHADIR CHACHOUA

KOCAELI 2024
KOCAELI UNIVERSITY

INSTITUTE OF SCIENCE

ELECTRONICS AND COMMUNICATIONS ENGINEERING DEPARTMENT

MASTER'S THESIS

APPLICATION OF AN OBJECT DETECTION-BASED PLC-CONTROLLED SORTING SYSTEM USING DELTA ROBOT

MOHAMMED ELKHADIR CHACHOUA

Assoc. Prof. Dr. Sıtkı ÖZTÜRK **Advisor, Kocaeli University.**	.....
Assoc. Prof. Dr. Oğuzhan KARAHAN **Jury Member, Kocaeli University.**	.....
Assoc. Prof. Dr. Melih KUNCAN **Jury Member, Siirt University.**	.....

Date the thesis was defended: December 13, 2024

ETHICS STATEMENT AND RESEARCH FUND SUPPORT

In this thesis/project, which I prepared in accordance with the thesis writing rules of Kocaeli University Institute of Science,

- I declare that this thesis/project is my own original work.
- I declare that I have acted in accordance with scientific ethical principles and rules in all stages of my study, including preparation, data collection, analysis, and presentation of information.
- I have cited all sources for the data and information obtained within the scope of this study and have included these sources in the bibliography.
- This study has been found to be compliant with the criteria set by the Institute of Science at Kocaeli University, using the plagiarism detection software subscribed to by the university.
- I have not falsified any of the data used,
- I have not submitted any part of this thesis/project as another thesis/project at this university or any other university.

I declare.

☒ No stage of this thesis/project work may be carried out by any institution/organization.It has not been supported with financial/infrastructure assistance.

☐ This thesis/project work in the scope of produced data AndinformationThis project was carried out with financial/infrastructure support from ..... under project number .....

I hereby declare that I accept all moral and legal consequences should any finding be contrary to this statement I have made regarding my work at any time.

Mohammed Elkhadir CHACHOUA

PUBLICATION AND INTELLECTUAL PROPERTY RIGHTS

I declare that I grant Kocaeli University permission to archive and make available, in printed and electronic formats, all or any part of my postgraduate thesis/project approved by the Institute of Science, under the conditions specified below. With this permission, all intellectual property rights other than the usage rights granted to the University will remain with me, and the use of all or part of my thesis/project in future studies (articles, books, licenses, patents, etc.) will belong to me.
I declare and undertake that this thesis/project is my own original work, that I have not infringed on the rights of others, and that I am the sole authorized owner of this thesis/project. I undertake that I have used copyrighted texts in my thesis, the use of which requires written permission from the copyright holders, with written permission, and that I will submit copies to the University upon request.
In accordance with the "Directive on the Collection, Organization, and Access to Graduate Theses in Electronic Format" published by the Council of Higher Education, my thesis will be made accessible on the YÖK National Thesis Center/Kocaeli University Libraries Open Access System, except for the conditions specified below.

☐ The institute's board of directors has decided to postpone the release of my thesis/project for two years after my graduation date.
☐ The institute's board of directors, in its reasoned decision, has postponed the release of my thesis/project for six months from my graduation date.
☒ No confidentiality order has been issued regarding my thesis/project.

Mohammed Elkhadir CHACHOUA

PREFACE AND ACKNOWLEDGEMENTS

In this study, a sorting system based on object detection using deep learning was implemented with a PLC-controlled Delta Robot.

I would like to express my gratitude and respect to my advisor, Assoc. Prof. Dr. Sıtkı ÖZTÜRK, for his invaluable contributions and guidance, as well as to my department professors whose knowledge and experience have helped me reach this point. I also extend my thanks to my dear family, who have supported me in every way and raised me.

I would like to express my special thanks to Mr. Abdülkadir YAYLA from the Department of Mechanical Engineering, Faculty of Engineering, Kocaeli University, for his support in making the mechanical modifications on the Delta robot.

December - 2024 Mohammed Elkhadir CHACHOUA

CONTENTS

ETHICS STATEMENT AND RESEARCH FUND SUPPORT.....i
PUBLICATION AND INTELLECTUAL PROPERTY RIGHTS.....ii
PREFACE AND ACKNOWLEDGEMENTS.....iii
CONTENTS.....iv
INDEX OF FIGURES.....vi
INDEX OF TABLES.....viii
SYMBOLS ANDINDEX OF ABBREVIATIONS.....ix
SUMMARY.....x
ABSTRACT.....xi
1. ENTRANCE.....1
2. LITERATURE REVIEW.....4
2.1. Method.....4
2.2. Results.....5
3. BACKGROUND INFORMATION.....9
3.1. Industrial Automation.....9
3.1.1. Robotic Guidance.....9
3.1.2. Quality control.....9
3.1.3. Machine Control.....10
3.2. Computer Vision.....10
3.2.1. Image Processing.....10
3.2.1.1. Image Display.....10
3.2.1.2. RGB Color Model.....10
3.2.1.3. HSV Color Model.....11
3.2.1.4. Grayscale Image.....11
3.2.1.5. Image Segmentation.....11
3.2.1.6. Thresholding.....12
3.2.1.7. Blurring.....12
3.2.1.8. Edge Detection.....13
3.2.1.9. Object Detection.....13
3.2.1.10.Region of Interest.....14
3.2.2. Camera Calibration.....14
3.2.2.1. Calibration Parameters.....15
3.2.2.2. Internal Parameters.....15
3.2.2.3. External Parameters.....15
3.2.2.4. Camera Calibration Models.....15
3.2.2.5. Pinhole Camera Model.....16
3.2.2.6. Fisheye Camera Model.....18
3.2.2.7. Types and Causes of Degradation Effects.....19
3.2.2.8. Radial Distortion.....19
3.2.2.9. Tangential Distortion.....19
3.3. Object Detection with Deep Learning.....20
3.3.1. Object Detection.....21
3.3.2. Deep Learning.....21
3.3.3. Object Detection Application with Deep Learning.....22
3.3.3.1. Gathering a Dataset of Images.....22
3.3.3.2. Adding a Label to the Dataset.....23

3.3.3.3. Preprocessing the Data Set....23
3.3.3.4. Training a Deep Learning Model....23
3.3.3.5. Model Evaluation....24
3.3.3.6. Using the Model for Object Detection....24
4. CONCEPTUAL DEFINITIONS IN ROBOTICS....25
4.1. Location Description....25
4.2. Orientation Definition....26
4.3. Homogeneous Transformation Matrix....30
4.4. Advanced Kinematics of Robotics....32
4.5. Inverse Kinematics of Robots....33
5. KINEMATIC MODEL OF THE DELTA ROBOT....34
5.1. DPR Advanced Kinematic Calculations....37
5.2. DPR Inverse Kinematic Calculations....39
5.3. Numerical Simulation and Validation of Kinematic Analysis....40
6. SYSTEM DESIGN....42
7. PLC-based servo motor position control.U....44
7.1. Setting up servo motor control with PLC....44
7.2. PLC and Servo Configuration in TIA Portal....46
7.3. Control Program Created in PLC....54
8. CAMERA CALIBRATION FOR VISUAL POSITIONING PURPOSES....55
8.1. Preparation of Calibration Images....55
8.2. Camera Calibration....56
8.3. Evaluation of Calibration Accuracy....56
8.4. Parameter Calculation....57
9. IMPLEMENTATION OF THE DELTA ROBOT SORTING SYSTEM....58
9.1. Homing and Camera Calibration....58
9.2. Object Detection Process with Deep Learning....58
9.2.1. Data Set Collection, Adding and Expanding Information Notes....60
9.2.2. Object Detection Model Training, Results and Evaluation....60
9.3. Visual Positioning....62
9.4. Collection Location Calculation Method....63
9.5. Communication Between Robotics and Computer Vision Systems....66
9.6. Integration of Computer Vision System with Delta Robot....67
10. RESULTS AND DISCUSSION....69
SOURCES....72
PERSONAL PUBLICATIONS AND WORKS....77
RESUME....78

INDEX OF FIGURES

Shape1.1. Schematic view and mechanical structure of the Delta robot.....2
Figure 3.1. HSV color space.....11
Figure 3.2. Segmenting a grayscale image using binary thresholds.....12
Figure 3.3. Example of different blurring methods.....12
Figure 3.4. Edge detection in binary images.....13
Figure 3.5. Example of vision with a computer detecting circles in an image.....14
Figure 3.6. Camera models.....16
Figure 3.7. Pinhole camera model.....17
Figure 3.8. Internal and external parameters.....17
Shape3.9. Real, camera and pixel coordinates.....18
Figure 3.10. Before and after the fisheye effect.....18
Figure 3.11. Effect of barrel and pincushion distortion.....19
Figure 3.12. Effect of tangential and radial distortion.....20
Figure 3.13. Illustration of the degradation effect.....20
Figure 3.14. Example of object detection.....21
Shape3.15. The difference between ML and DL.....22
Shape4.1. Vector representation between and$\{A\}\{B\}$.....26
Shape4.2. Two CCSs with the same center but different orientations.....27
Shape4.3. Representation of two concentric CCSs.....29
Shape4.4. Graphical representation of equation (4.13).....29
Shape4.5. Two CCSs with different orientations and centers at different points.....30
Figure 4.6. Definition of point in terms of different CCSs.P.....31
Figure 4.7. Advanced kinematic representation.....32
Shape4.8. Inverse kinematic representation.....33
Shape5.1. Delta Parallel Robot.....34
Shape5.2. Assignment of coordinate systems.....35
Shape5.3. Forward and reverse kinematic verification graph.....40
Figure 5.4. X, Y and Z Position Error.....40
Figure 5.5. 3D view of the Delta study area.....41
Figure 5.6. Top view of the Delta study area.....41
Figure 5.7. Front view of the Delta study area.....41
Figure 6.1. Designed robotic system and control panel.....42
Figure 6.2. Components of the realized system.....43
Figure 7.1. Equipment used in the robotic system.....45
Figure 7.2. PLC controller and servo drive PNP cable connection.....45
Figure 7.3. Adding a PLC device to the project.....46
Shape7.4. PLC Device Configuration.....47
Figure 7.5. Selection of the PTO signal.....47
Figure 7.6. Adding Axis PTO technology object.....48
Figure 7.7. Axis PTO technology object configuration.....48
Figure 7.8. Adjustment of mechanical and dynamic parameters.....49
Figure 7.9. Axis Commissioning process.....50
Figure 7.10. MC_POWER instruction block.....51
Figure 7.11. Graph of the MC_POWER instruction block.....51
Figure 7.12. MC_HOME command block.....52
Figure 7.13. MC_MOVEABSOLUTE command block.....53

Figure 7.14. Graphic of the MC_MOVEABSOLUTE command block.....53
Figure 7.15. Powering the Servo Axes.....54
Figure 7.16. Home Servo Axes.....54
Shape7.17. Moving the Servo Axes.....54
Figure 8.1. Sample images used in calibration.....55
Figure 8.2. Calibration process.....56
Figure 8.3. Projection error graph.....56
Figure 8.4. Code for extracting internal and external parameters.....57
Figure 9.1. Single-stage and two-stage object detection algorithms.....59
Figure 9.2. Architecture of single-stage and two-stage models.....59
Figure 9.3. Examples from the collected dataset.....60
Figure 9.4. YOLOv2 and SSD Learning Loss Graph.....61
Figure 9.5. YOLOv2 and SSD Learning RMSE graph.....61
Figure 9.6. Accuracy/Recall graph of YOLOv2 and SSD.....62
Figure 9.7. Results of the YOLOv2 model on test images.....62
Figure 9.8. Visual positioning results.....63
Shape9.9. Code execution time for different trials.....64
Figure 9.10. Path created for the pick-and-drop operation.....65
Figure 9.11. Communication interface of the implementing system.....67
Figure 9.12. Steps in the object sorting process.....67
Shape9.13. Pseudocode of the implementing system.....68

INDEX OF TABLES

Table 5.1. Parameters of the delta robot used.....34
Table 5.2. Fixed platform coordinate systems.....35
Table5.3. Coordinate systems of the middle joints of the arms.....36
Table5.4. Mobile platform coordinate systems.....36
Table 7.1. Pin assignment for Pulse output, Direction output, and Activation output.…47
Table9.1. , and calculated values for $x1$ $x2$ $x3$.....65
Table9.2. Data values for obtaining the first objective.....66
Table9.3. Data values for achieving the second objective.....66
Table 9.4. Data values for achieving the third objective.....66

INDEX OF SYMBOLS AND ABBREVIATIONS

K	Camera Calibration Internal Matrix
P	Camera Matrix
R	Camera Calibration External Matrix
r	Platform Radius (mm)
T	Transformation Matrix
t	Time(s)
γ	Angle between the angle bisectors of a triangle (rad)
Δ	Discriminant of a Quadratic Polynomial Equation
θ	Robot Joint Angle
ν	Speed (mm/s)
ω	Scale Factor

Abbreviations

2D	Two Dimension
3D	Three Dimensions
AP	Average Precision
AI	Artificial Intelligence
COCO	Common Objects in Context
CCS	Cartesian Coordinate System
DPR	Delta Parallel Robot
DL	Deep Learning
DOF	Degree of Freedom
HSV	Hue Saturation Value
HR	Advanced Kinematics
LED	Light Emitting Diode
ML	Machine Learning
MC	Motion Control
OPC	Open Platform Communication
PLC	Programmable Logic Controller (PVC)
PTO	Pulse Train Output
PET	: Polyethylene Terephthalate (Polyethylene Terephthalate)
PNP	Positive-Negative-Positive
PWM	Pulse Width Modulation
PR	Precision Recall
ROI	Region of Interest
RGB	Red Green Blue
ResNet	: Residual Network
RMSE	: Root Mean Sqaure Error
SSD	Single Shot Detection
TK	Inverse Kinematics
VOC	Visual Object Classes
YOLO	You Only Look Once

APPLICATION OF AN OBJECT DETECTION-BASED PLC-CONTROLLED SORTING SYSTEM USING DELTA ROBOT

SUMMARY

The main objective of this study is to improve the operation of a computer vision-assisted system using a PLC-controlled delta robot to separate polyethylene terephthalate (PET) objects on a waste line on a moving conveyor belt. For the computer vision part of this separation application, images are obtained using a camera connected to the system. These images are then processed by a program that uses a deep learning model for object detection, and the real-time position coordinates of each detected object are determined using camera calibration parameters. A position calculation method is proposed to calculate the real-time position of moving objects on the conveyor belt. This method is used to provide the updated position of the objects on the conveyor. A delta robot is used to pick up the moving objects. This robot is controlled by a programmable logic controller (PLC). Kinematic model analysis to move the delta robot to the desired position is performed in the MATLAB environment. The obtained position information is sent to the PLC using the Open Platform Communication Protocol (OPC). Based on the received values, the delta robot moves to the instantaneous position of the target object, picks up the object from the conveyor, and transports it to the separation area. The results obtained from this application support the feasibility of integrating artificial intelligence algorithms with automated systems. Similarly, it has shown that it can be used optimistically in the industry to obtain faster and more accurate results.

Keywords: Computer Vision, Delta Robot, Deep Learning Application, Object Detection, Automation and Control.

DELTA ROBOT SORTING SYSTEM CONTROLLED WITH PLC BASED ON OBJECT DETECTION

ABSTRACT

The focus of this research is to develop an artificial intelligence (Ai) powered computer vision-guided system using a delta robot to separate the polyethylene terephthalate (PET) bottles on a moving conveyor belt. For the computer vision part, images are acquired using a camera and processed by a program that uses deep learning algorithms for object detection, Then the real-world position coordinates of each detected object are found using the computed camera calibration parameters. To estimate the real-time location of moving objects on the conveyer belt, a picking position calculation method is proposed. To pick up the moving objects, a delta robot is used. This robot is controlled with a programmable logic controller (PLC). To move the delta robot to the desired location, the kinematic model analysis is performed. The obtained values are then sent to PLC using the Open Platform Communications protocol (OPC). Based on the received values, the delta robot picks the target objects and places them into the sorting area. The results obtained from this application support the applicability of integrating AI algorithms with automated systems. In the same way, it showed that it can be optimistically used in the industry to obtain faster and more accurate results.

Keywords: Computer Vision, Delta Robot, Deep Learning Application, Object Detection, Automation and Control.

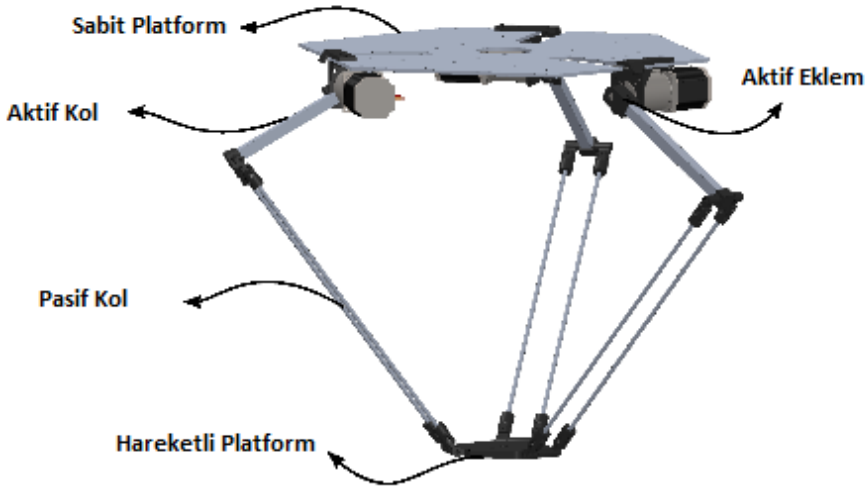
1. INTRODUCTION

Since the first industrial revolution in the mid-18th century, engineers have strived to propel industry toward higher productivity through new technologies; the first revolution was driven by steam power, the second by electric power, and the third by electronic power. Today, with the continued rise of Industry 4.0, a revolution based on smart technology, the goal is to achieve greater productivity with less human intervention. As in previous industrial revolutions, engineers are exploring new ways to expand and further advance towards this goal. One area being researched is computer vision and deep learning; these will become increasingly important in the near future with more sophisticated cameras, higher bandwidth, and faster calculations.

Traditional factories use manual picking and placing for sorting, which is a lengthy process with less accurate results (Lin et al., 2016). It also consumes a significant amount of human resources. To improve accuracy and stability in sorting tasks, a more cost-effective system is proposed: an automated computer vision-assisted robotic system using a delta parallel robot.

The Delta Parallel Robot (DPR) was designed by Professor Reymond Clavel in the 1990s. This robot won the Gold Robot Award in 1999 and production began that same year by Asea Brown Boveri (ABB) (Clavel, 1991). Many types of delta robots differ structurally and functionally (Gasparetto et al., 2019). Delta robots are becoming increasingly important in the Industry 4.0 field due to their ability to move quickly and precisely. They can be operated using industrial control systems such as programmable logic controllers (PLCs). Thanks to various integrated camera applications, the parallel delta robot is widely used in industry for product sorting, sequencing, selection, transfer, 3D printing, and packaging systems (Bonev, 2001).

As shown in Figure 1.1, delta robots are a type of parallel (closed loop) robot (Gugliementi, 1994). These robots have many advantages and disadvantages compared to serial (open loop) robots, one of the biggest disadvantages being their limited usable workspace. However, parallel manipulators also offer other advantages such as the continuity or higher rigidity of joint positioning, as well as high accuracy (Nzue et al., 2013).



Shape1.1.Schematic view and mechanical structure of the Delta robot.

The main objective of this study is to develop a computer vision-assisted system using a delta robot to separate target objects (polyethylene terephthalate (PET) objects) from a waste line on a moving conveyor belt. For the computer vision part of this separation application, images are obtained using a camera connected to the system. These images are then processed by a program that uses a deep learning model for object detection, and the real-time position coordinates of each detected object are determined using camera calibration parameters. An object pickup position calculation method is proposed to calculate the real-time position of the moving objects whose positions are determined on the conveyor belt. This method is used to provide the current position of the objects on the conveyor. A delta robot is used to pick up the moving objects from the belt. This robot is controlled by a PLC. Kinematic model analysis is performed to move the delta robot to the desired position. The obtained values are then sent to the PLC using the OPC protocol. Based on the received position values, the delta robot moves to the instantaneous position of the target object, picks it up from the conveyor, and transports it to the separation area.

This thesis is structured as follows. The next section provides a literature review of previous studies that contributed to this thesis. Chapter 3 presents theoretical information on industrial automation, computer vision, deep learning, and object detection. Chapter 4 explains conceptual definitions in robotics.

Chapter 5 describes the kinematic model analysis where coordinate systems are assigned and the relevant symbols are given. Chapter 6 presents the forward and inverse

kinematic analysis of the delta parallel robot. Following this, the resulting equations of the kinematic model are used for numerical simulation and evaluation. Chapter 6 also covers the hardware structure of the system, detailing the components and system operation modules used. Position control of servo motors via PLC is presented in Chapter 7. Chapter 8 addresses the camera calibration process for visual positioning. Chapter 9 designs the implementation of a computer vision system for the delta robot to pick up target objects and transport them to the sorting area. Object detection using deep learning algorithms is also emphasized. Following this, visual positioning is performed using the obtained camera calibration parameters. Finally, the method for calculating the collection position is presented. The last chapter shows and discusses the results and includes suggestions for future studies.

In this study, visual processing, positioning, and motion planning codes were written using the MATLAB programming language.

2. LITERATURE REVIEW

This section will present the methods and results of a current literature review on the use of deep learning for computer vision in industrial automation.

2.1. Method

A literature review was conducted to identify current uses of computer vision and deep learning in relevant applications.

This research was conducted by searching for articles to find studies that met the following criteria:

- The use of computer vision or deep learning for sensing purposes during its implementation.
- Working on an application in any of the areas of automation such as robotic guidance, quality control, and machine control.
- It must be explicitly stated that an application of computer vision and deep learning-assisted robotics systems is being used.

Four main platforms were used for the article search, but not limited to: IEEE Xplore, Google Scholar, Springer Nature, and Elsevier. The search was performed in the following steps:

- Keywords were searched for in both the title and the full text. Keywords used, though not limited to, include “computer vision”, “delta parallel robot”, “deep learning application”, and “computer vision-assisted robots”.
- The titles of the search results were evaluated to determine whether they met the criteria.
- When the title was deemed appropriate, it was evaluated to determine whether the relevant article abstract met the criteria.
- Where the article abstract was deemed appropriate, a keyword search and a brief review of the full text were conducted.
- If the entire article is relevant to the topic, it has been summarized and presented in the conclusion section.

2.2. Results

This section presents the results obtained from the literature review. The research findings, obtained by following the steps described earlier, are summarized and listed chronologically by publication year.

A study conducted at the Polytechnic University of Valencia used real-time vision to guide robots on a simulated factory floor (Zaera et al., 2001). The experiment utilized a ceiling-mounted camera to provide a top-down view of the entire floor, which included both mobile robots and distributed, immovable machinery. The robots' task was to supply products to and collect products from the machines. Real-time constraints were incorporated into the system to ensure a collision-free operation. Using real-time vision, a system was achieved in which a central unit successfully guided the robots without collisions (Zaera et al., 2001).

A study was conducted at the Catholic University of San Pablo to automate the chestnut sorting process. The experiments utilized a real-time computer vision system with a camera positioned to view two production lines from above. The production lines featured two cylinders that rotated the chestnuts as they passed, allowing the camera to capture a 360-degree view of each chestnut. Typical defect characteristics, such as cracked, sour, or chipped nuts, were then classified and discarded. The resulting system yielded highly positive results with an 87% defect detection accuracy (Alvarez et al., 2014).

A study was conducted at RWTH Aachen University in Germany to develop tools and concepts for real-time quality control of non-crimp fabrics (Schmitt et al., 2015). The study aimed to provide a solution capable of presenting its results at a deterministic rate; this is something not available on the market for optical measurement systems at the time of the study. The system examines the fabric and detects the position and orientation of the fiber to determine whether it meets the requirements. All combinations examined had an uncertainty level below 0.3, which is below the maximum measurement uncertainty criteria for industrial use. Furthermore, the study investigated how different factors affect measurement uncertainty and concluded that

material type is the most critical parameter affecting random-based error, while aperture, resolution, filter algorithm, and material type have the greatest impact on systematic error (Schmitt et al., 2015).

In 2016, Jinan University, in collaboration with Beijing Normal University, designed a real-time steel bar recognition and counting system (Zhao et al., 2016). The system used a high-speed camera to capture images of steel bars resting on a chain bearing, and this image was transmitted to an IPC (Integrated Counterpart). The IPC applied various image processing techniques to count the number of bars and sent a stop signal to the conveyor belt when the count reached a predetermined threshold. To meet real-time requirements, different image processing algorithms were compared to find the best in terms of speed and accuracy. The final design showed an accuracy rate of 96% for bars larger than 15 mm and 94% for bars smaller than 15 mm (Zhao et al., 2016).

In 2016, Chih-Jer Lin et al. proposed a vision-assisted robotic system using a delta parallel robot to pick up and position moving workpieces on a conveyor belt according to their shapes. This system utilized Sobel and Canny edge detection techniques in its image processing. Experimental results showed that the Canny edge detection method performed better than the Sobel method in this case study (Lin et al., 2016).

A PC-centric delta parallel manipulator with real-time computer vision capability was presented at the 2018 WRC Advanced Robotics and Automation Symposium (Liu et al., 2018). The system used a delta robot to sequence circles from one conveyor belt to another, using images to determine the position of the circles. The real-time image acquisition and processing time had a time distribution between 12.1 and 13 ms, achieving sufficient real-time performance. The resulting system was able to acquire and place circles 120 times per minute with a conveyor belt speed of 0.2 m/s, an average horizontal distance of 30 cm, and a lifting distance of 20 cm. The controller had the largest control latency at 35.43 µs, and the vision system had a processing capacity of up to 75 frames per second (Liu et al., 2018).

A publication presented at the 2018 International Conference on Computer Science and Engineering (ICU) described the use of computer vision for quality control in e-jet printing for micro-scale manufacturing (Lies et al., 2018). The authors proposed a

system in which a real-time imaging system is mounted to horizontally view an e-jet to analyze the flow from the nozzle to the workpiece. The study demonstrated that the real-time imaging system yielded impressive results for monitoring and real-time quality control and could be further utilized in a feedback control system for advanced e-jet printing (Lies et al., 2018).

In 2019, a study on the use of computer vision and deep learning for a pick-and-drop robot was conducted at the University of Pahang, Malaysia (Farag et al., 2019). The study utilized a SCARA robot and a real-time computer vision system with a deep learning model trained to recognize a cylindrical object. The camera was positioned with a top-down view of the workspace to enable the robot to move the object from the grasping area to the placement area. The results of 50 experiments showed that the system achieved a 100% success rate for both pick-and-drop and placement operations with a detection time of 0.38 seconds (Farag et al., 2019).

A study conducted at the University of Maribor investigated a plastic granule separation system (Persak et al., 2020). The system used real-time computer vision to analyze transparent plastic granules passing through a transparent, backlit conveyor belt. The aim was to remove defective granules at the end of the conveyor belt using compressed air. Real-time computer vision was used to achieve the sufficiently high performance required for sorting. The resulting system had an average contamination rate of only 0.94% in the accepted material. Granule image analysis, which classified defective and accepted granules, had 100% accuracy, meaning it could theoretically perform the sorting process with 100% accuracy. Therefore, any errors occurring during sorting were a result of other factors such as granules sticking to the conveyor and the coherence forces between the granules (Persak et al., 2020).

In 2022, Yuh Wen Chen et al. reported using additive manufacturing to design an automated optical inspection system using You Only Look Once deep learning models (Chen et al., 2022). They demonstrated an average accuracy rate of over 70% for detecting real-time video data on production lines. The authors also conducted a comparative analysis of classification efficiency among different YOLO algorithms. Their work on optical inspection showed that manual inspection significantly reduces human labor costs in the electroplating industry and is promising for automated

production (Chen et al., 2022).

Vo Duy Cong et al. (2023) developed a system using a parallel delta robot to classify objects by their color using image processing techniques (Cong et al., 2023). The DPR used is driven by DC servo motors and controlled by an Arduino and a Raspberry Pi 4. The latter was used to perform image processing on objects for color-based classification. The results showed that the system's accuracy was 100% for classifying red and yellow objects and 97.5% for classifying green objects. Furthermore, the entire process took an average of 1.8 seconds to sort each object (Cong et al., 2023).

3. BACKGROUND INFORMATION

This section provides a brief introduction to fundamental concepts related to industrial automation and computer vision. It covers camera calibration methods for visual positioning, as well as object detection using deep learning.

3.1. Industrial Automation

Industrial automation is the process of coordinating mechanical, hydraulic, electrical, computational, chemical, biochemical, and/or biological units to produce a specific product. This process is automated through the integration of measurement, control, and information technologies to operate and control the complex systems and processes involved in production. Within the field of automation, there are various areas that utilize computer vision applications, such as robotic guidance, quality control, and machine control. These areas are discussed in more detail in later sections.

3.1.1. Robotic Guidance

Robotic guidance refers to the steering of both robotic arms and mobile robots. For a robot to navigate an environment, it needs information about its position relative to nearby objects (Courtney et al., 1984). This information is obtained through sensors such as radars, ultrasonic rangefinders, and computer vision (Arreguin, 2008). Data from these sensors can be transmitted to the control system to calculate the position of the robot and/or object relative to its surroundings. A typical application within robotic guidance is path planning (Lavalle, 2006). The aim is to find the shortest and collision-free path between two points. This allows the robot to optimize its movement, enabling it to move at high speed from one workpiece to another, and at a slower and more precise speed as it approaches the object.

3.1.2. Quality control

Quality control is the process of verifying that manufactured products meet the quality requirements specified by the manufacturer. Many methods are used for this purpose, but a common approach is to measure the product in some way (e.g., measuring dimensions, angles, and surface roughness). The traditional way of quality control is to manually check a sample in a product batch to ensure the quality of the entire batch.

This method is flawed because, depending on the selected sample, it can lead to the rejection of high-quality products or the passage of low-quality products. Therefore, it is beneficial to perform quality control of each product using computer vision, which should be done in parallel with the production process. By implementing this quality control automation, the manufacturer reduces quality control time, ensures that each product is of high quality, and increases efficiency in the system (Tiwari, 2016).

3.1.3. Machine Control

Machine control generally describes the process of controlling a machine or process based on sensor feedback, typically in an industrial setting. The controller's objective is to bring any type of actuator to a specific state based on the given feedback. This ranges from speed control of a conveyor belt with an incremental encoder as feedback to solenoid operation for the release of a substance in a chemical process with temperature as feedback.

3.2. Computer Vision

3.2.1. Image Processing

3.2.1.1. Image Display

This section explains what an image is and how it can be represented. An image consists of a 2-dimensional array of pixels, where each pixel element defines light at a specific location in the image. There are different formats used to represent a pixel, and below are descriptions of three of the most commonly used formats in computer graphics.

3.2.1.2. RGB Color Model

The RGB color model is a three-channel color model where each pixel has a value for red, green, and blue. These three colors are then combined in varying ratios to obtain a specific color. The RGB color model is widely used in display technologies because a pixel typically consists of one red, one green, and one blue LED, making it simple to change the intensity of each LED. The disadvantage of this model is its susceptibility to uneven lighting and nonlinear relationships between colors (Yu et al., 2020).

3.2.1.3. HSV Color Model

The Hue-Saturation-Value (HSV) color model was introduced in the 1970s by researchers aiming to design a color model that interpreted colors more similarly to how humans perceive color (Joblove, 1978). Unlike the RGB model, the HSV model separates brightness and hue, making it less sensitive to uneven lighting and providing a more accurate color definition. HSV is typically represented in a conical color coordinate system, see Figure 3.1. Hue is the angle around the central axis and represents the dominant color. Saturation is the radius and indicates how accurate the color is, i.e., how far it is from gray. The brightness value expresses the brightness of the color and ranges from 0 to 100, where 100 corresponds to full color and 0 to pure black.

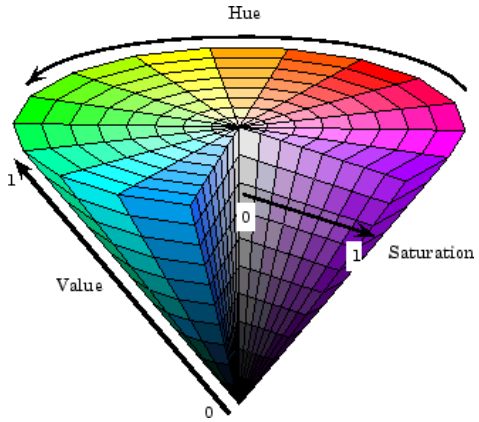


Figure 3.1.HSV color space

3.2.1.4. Grayscale Image

A grayscale image is a single-channel model where the pixel value changes in intensity from black to white. This means that, compared to RGB and HSV models, each pixel has only one value. It is often used to locate a specific feature in an image.

3.2.1.5. Image Segmentation

Image segmentation is the process of converting any representation of an image, such as a grayscale image, into a binary image where each pixel can take a value of 0 or 1. This is done to simplify the image and highlight areas of interest, creating an image that is easy for a computer to analyze. This process is one of the most important in computer vision, as a perfect binary image is key to accurate object detection, edge finding, and

other image analysis.

3.2.1.6. Thresholding

Thresholding is a very common method of dividing an image into segments. The basic concept is to set a pixel threshold where all pixels exceeding the value are set to black or white, and all pixels below the threshold are set to the opposite color (see Figure 3.2). This segmentation method is based on the assumption that there is a significant difference in pixel density between the object of interest and its background (Qingmao et al., 2006).

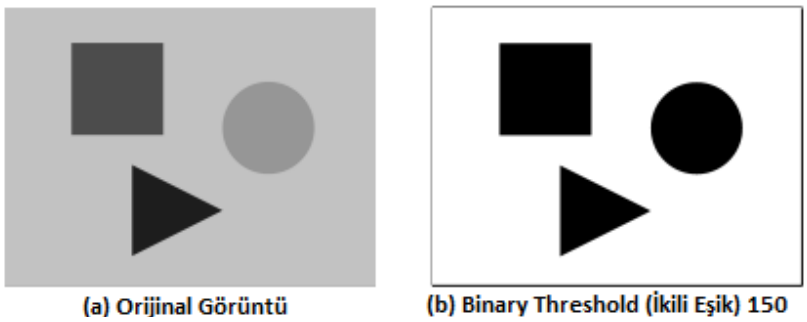


Figure 3.2.Segmenting a grayscale image using binary thresholds.

3.2.1.7. Blurring

In many cases, blurring can be a useful method for removing noise from an image and can be used to make an image more suitable for object detection. It can create a smoother shape by smoothing the pixels at the edges of an object. Three of the most common filtering methods are bilateral-, Gaussian-, and median-blurring methods, shown in Figure 3.3.

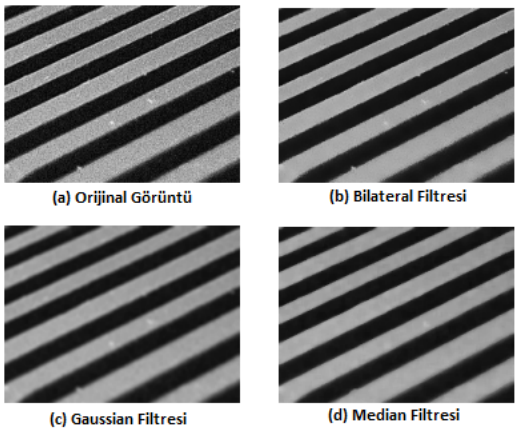


Figure 3.3.Examples of different blurring methods.

3.2.1.8. Edge Detection

Edge detection aims to find edges in an image by using algorithms to identify contrasts where there is a sharp difference in pixel density. For example, it can be used to detect a car license plate, identify a fingerprint, or find various shapes as shown in Figure 3.4. There are a wide variety of algorithms to choose from, but three of the most commonly used are Prewitt, Sobel, and Canny (Hagara et al., 2020).

3.2.1.9. Object Detection

Object detection is the concept of identifying objects in a digital image, such as cars, people, or a workpiece. It combines localization with classification by aiming to find the characteristics of a particular object, such as its shape and color (Solovyev, 2021).

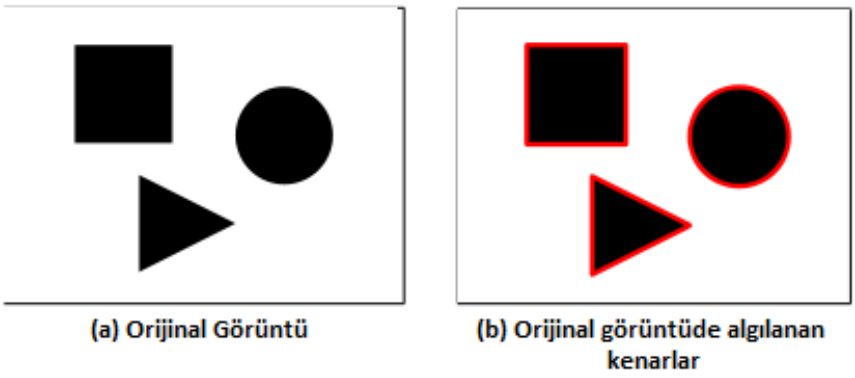


Figure 3.4.Edge detection in binary images.

Finding an object typically involves a series of steps. Let's say the goal is to find a circle in a target image.

The first step is to convert the image to a grayscale image. The grayscale image is then

split into two and passed through an edge detection algorithm. Edges that form a loop and enclose an area are saved to a list of potential objects. The final step is to examine all objects enclosed by the edge detection and compare their attributes to a set of criteria that characterize a circle (e.g., circularity). The remaining objects are shown, and the process is complete. The sequence is visualized in Figure 3.5.

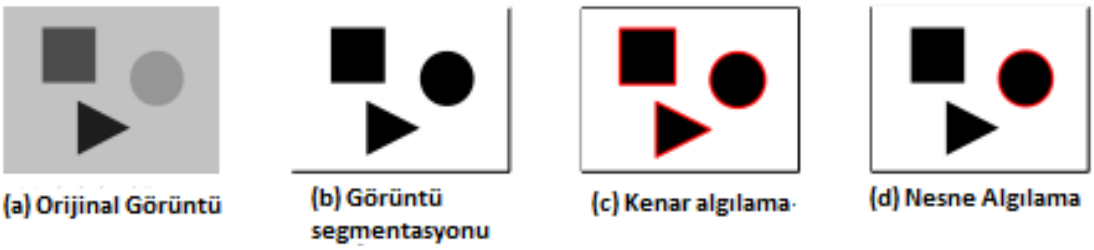


Figure 3.5.An example of a visual sequence with a computer that detects circles in an image.

3.2.1.10. Region of Interest

A common practice in computer vision is to define a region of interest (ROI) for the image to be processed. Defining an ROI means selecting a region within the image where image processing will be performed. Pixels outside the ROI are excluded from processing. By defining an ROI that includes only the relevant parts of the image, performance can be improved because calculations are performed on fewer pixels.

3.2.2. Camera Calibration

Camera calibration is a fundamental component of computer vision and a crucial process in many different applications, including 3D visualization, 3D modeling, object tracking, AR (Augmented Reality), and others. Accurate calibration ensures precise measurements and reliable analysis by correcting distortions and calculating intrinsic and extrinsic camera parameters. This section discusses the methods and algorithms used in the calibration process. It also covers the process of extracting intrinsic and extrinsic (rotation and translation) matrices based on various camera models, and includes an example of distortion effects and their various types.

The function of the camera is to capture a three-dimensional (3D) image of the world scene and convert it into a two-dimensional (2D) image. As shown in Equation (3.1), the equation represents the camera equation, where is the point of the two-dimensional

image, is the camera matrix, and is the three-dimensional world point.$x = P\,X x P\,X$

$$x = P\,X \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{3.1}$$

Camera calibration is a widely used process in image processing. It aims to calculate properties that relate three-dimensional objects to their projection onto a two-dimensional plane. This process is commonly used in applications requiring the calculation of distances from an image. In these applications, the camera typically needs to be classified according to intrinsic parameters such as axis skew, focal length, and principal point, while orientation is defined by extrinsic parameters such as rotation and translation. The methods used in the calibration process to find camera parameters (matrices) are based on finding a relationship between three-dimensional points on the world scene and their counterparts in the two-dimensional image.

3.2.2.1. Calibration Parameters

The camera calibration process consists of calculating camera parameters, which are represented by three matrices. Camera calibration matrices are technically defined by estimating the characteristics of a camera. The purpose of deriving the three fundamental matrices of internal and external parameters is to establish a relationship between three-dimensional points in the real world and the equivalent two-dimensional points in the image captured by the calibrated camera. This relationship is then used to calculate real-world distances (URL-1).

3.2.2.2. Internal Matrix

The internal matrix links pixel coordinates to camera coordinates of the captured image, such as the lens's optical center, focal length, and radial distortion coefficients.

3.2.2.3. External Matrices

External matrices (Translation and Rotation Matrices) are used to determine the camera orientation and the position of the scene relative to the world coordinate system.

3.2.2.4. Camera Calibration Models

A camera model is the mathematical relationship between the real-world coordinates of a point in a light-projected scene and the two-dimensional coordinates projected onto the image plane (URL-2).

The pinhole camera model, or perspective camera model, is the first model followed as a reference and is also called the ideal camera model, but there are many other camera models, including the fisheye camera model (see Figure 3.6).

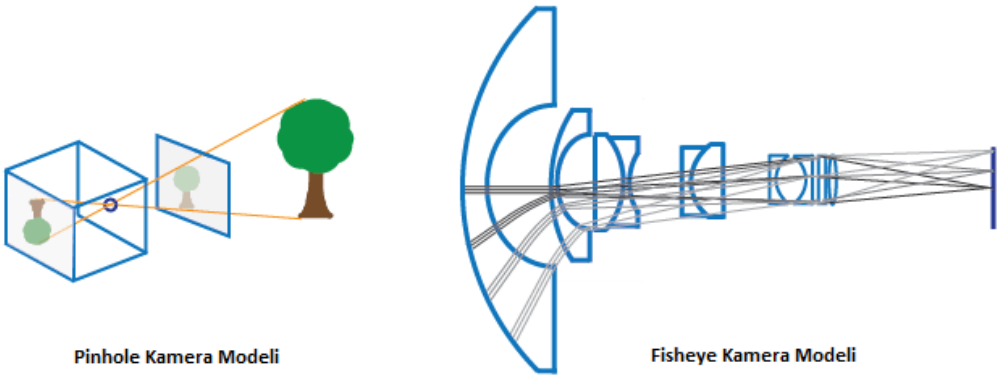


Figure 3.6.Camera models

The pinhole calibration algorithm is based on Jean-Yves Bouguet's (Bouguet, 2003) model. This model incorporates the pinhole camera model (Zhang, 2016) and lens distortion (Heikkila, 1997). The pinhole camera, as previously defined, is the ideal camera model. It is so named because the model algorithm does not include a lens, and therefore the distortion effects of the lens on the image are not considered during the calculation of the parameters. This can lead to less accurate results, unlike the fisheye model, which includes these effects.

For a mathematical representation of a real camera simulation, lens distortions that may occur in the image must be taken into account. These distortions include radial and tangential lens distortion.

3.2.2.5. Pinhole Camera Model

As shown in Figure 3.7 below, the working principle of an ideal camera model is that light rays reflected from a real object in the three-dimensional world pass through the camera's focal point and are reflected back to the other side of the camera's image plane.

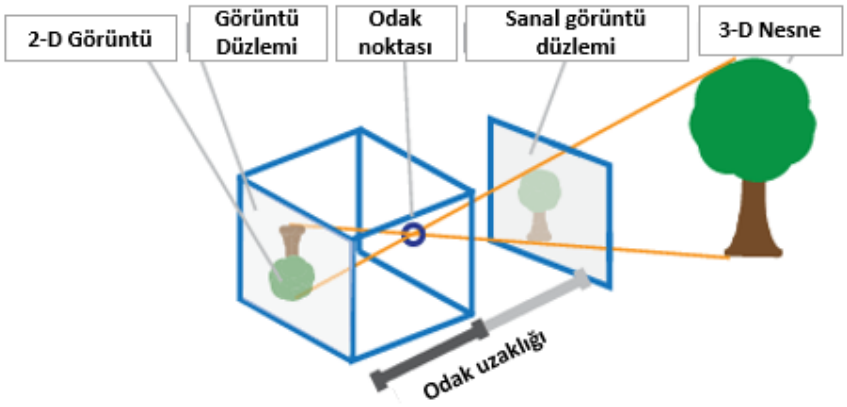


Figure 3.7.Piinhole camera model

After calibrating the camera, the calculation of three fundamental matrices is performed. The matrix K represents the internal parameters. The rotation matrix R and the translation matrix t represent the external parameters. The camera matrix P is calculated as shown in equation (3.3). The coefficient is the scale factor of the captured image. This is then substituted into equation (3.2), which is a shortened version of equation (3.1) mentioned earlier. Equation (3.2) represents the mathematical relationship between the coordinates of a three-dimensional point in the real world and a two-dimensional point in the plane of the captured image. Figure 3.8 below shows the transformations between the points shown using the obtained matrices.*w*

$$w\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = P\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{3.2}$$

$$P = \begin{bmatrix} R \\ t \end{bmatrix} K \tag{3.3}$$

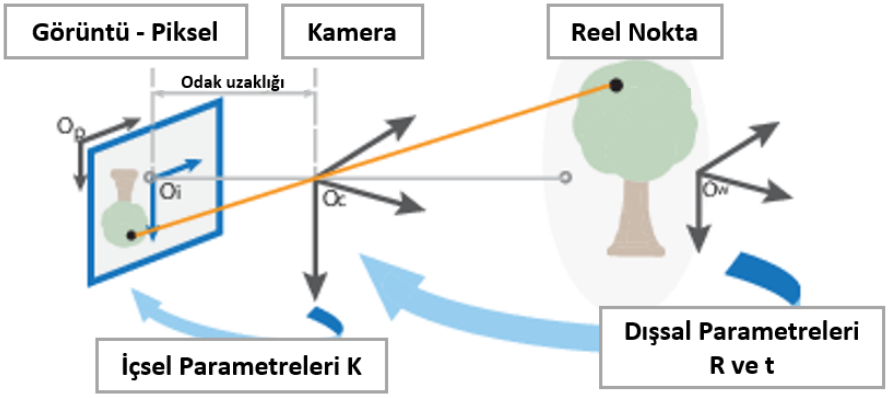


Figure 3.8.Internal and external parameters

3.2.2.6. Fisheye Camera Model

As previously defined, camera calibration is the process of calculating the camera matrix P by extracting the necessary matrices, namely the rotation and translation matrices R and t, and the internal matrix K. These are then used to calculate the two-dimensional coordinates in the image from the three-dimensional coordinates of the real world by tracing them through the camera coordinates, as shown in the diagram in Figure 3.9.

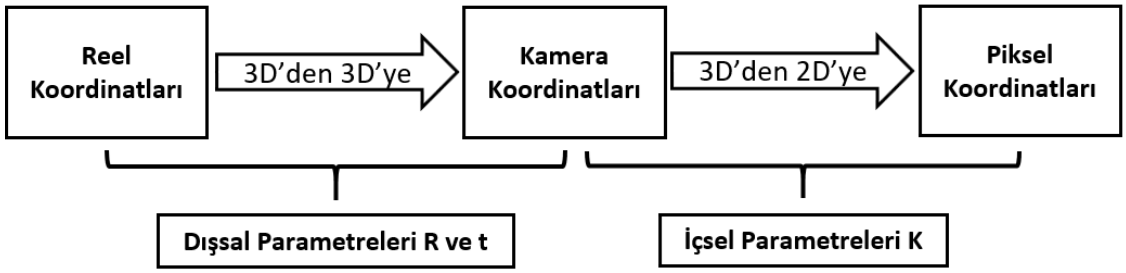


Figure 3.9. Real, camera and pixel coordinates.

The pinhole camera, as previously described, is the ideal camera model. It is so named because the model's algorithm does not include a lens, and therefore the effects of lens distortion on the image are not considered when calculating the camera matrix P. To simulate a real camera, lens distortions that can affect the image must be taken into account.

The fisheye camera model is more accurate than the ideal model because its algorithm includes a lens, making it a more accurate model. MATLAB allows us to use its calibration mechanism for both ideal and fisheye models (URL-3). Figure 3.10 shows an example of the before and after image of the distortion created by the fisheye model.



Figure 3.10.Fisheye effect before and after

Fisheye cameras, in addition to optically assisting odometry, help solve the challenges of simultaneous localization and mapping. Other applications include surveillance cameras in places such as airports and parking lots, underwater filming, night photography, image fusion, and virtual reality applications. These cameras enable the capture of wide panoramic or hemispherical images by using a complex array of lenses to increase the camera's field of view. The specific type of imaging in the aforementioned applications is achieved by applying specific distortions at the level of the image's perspective lines, providing a wide field of view.

3.2.2.7. Types and Causes of Distortion Effects

Better images can be obtained when using a lens, but the lens can produce some distortion effects (URL-4). Distortion effects are classified into two types:

3.2.2.8. Radial Distortion

Radial distortion is a distortion in a two-dimensional image that occurs when light rays reflected from a real-world three-dimensional object onto a camera lens bend. As shown in Figure 3.11, there are two types of radial distortion. Negative distortion, or Barrel distortion, occurs when the light ray bends in the negative direction (outward). Positive distortion, or pincushion distortion, occurs when the light ray bends in the positive direction (inward).

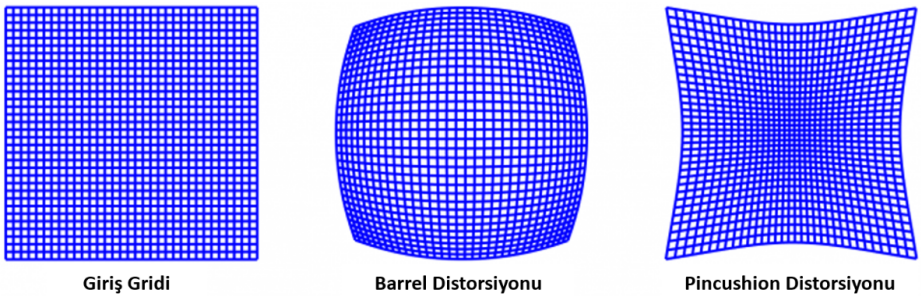


Figure 3.11.Barrel and pincushion distortion effect.

3.2.2.9. Tangential Distortion

The second type of distortion effect an image can experience when using a camera lens is tangential distortion. This distortion is caused by the angle between the lens and the camera sensor (URL-5). As a result, the image appears tilted and distorted.

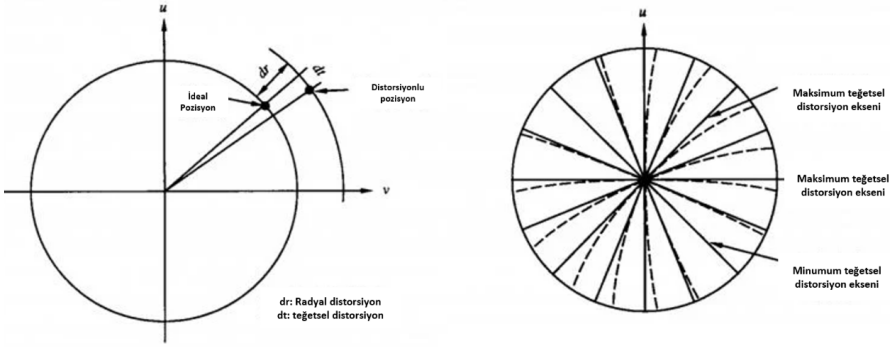


Figure 3.12.Effect of tangential and radial distortion.

Figure 3.12 shows diagrams illustrating the effects of tangential and radial distortion. Solid lines represent no distortion, while dotted lines represent tangential distortion (left part of the figure). The right part of Figure 3.12 shows an example of a pixel displaced from its ideal position due to tangential distortion in the image, as shown.

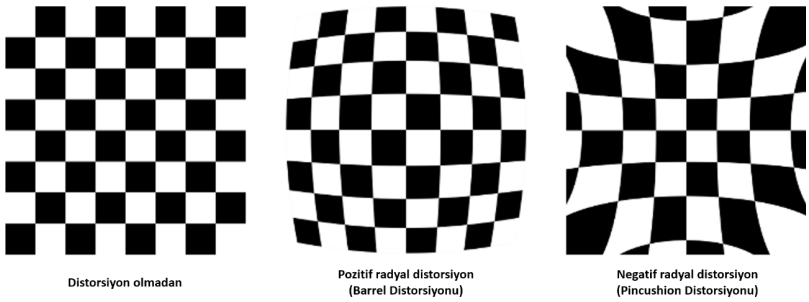


Figure 3.13.Demonstration of the distortion effect.

Figure 3.13 shows both positive and negative radial distortion, referred to as barrel

distortion and pincushion distortion.

3.3. Object Detection with Deep Learning

Compared to traditional methods of identifying objects in an image using image processing, the process of identifying objects using deep learning algorithms yields faster and more accurate results. This is because deep learning algorithms rely on determining the features of images based on the target objects, training parameters, and an image set. Well-known deep learning algorithms include Faster R-CNN, You Only Look Once (YOLO), and Single Shot Detection (SSD). Object detection is applied in various fields such as image classification and autonomous driving vehicles and surveillance. Figure 3.14 below shows an example of multi-class object detection using deep learning algorithms.

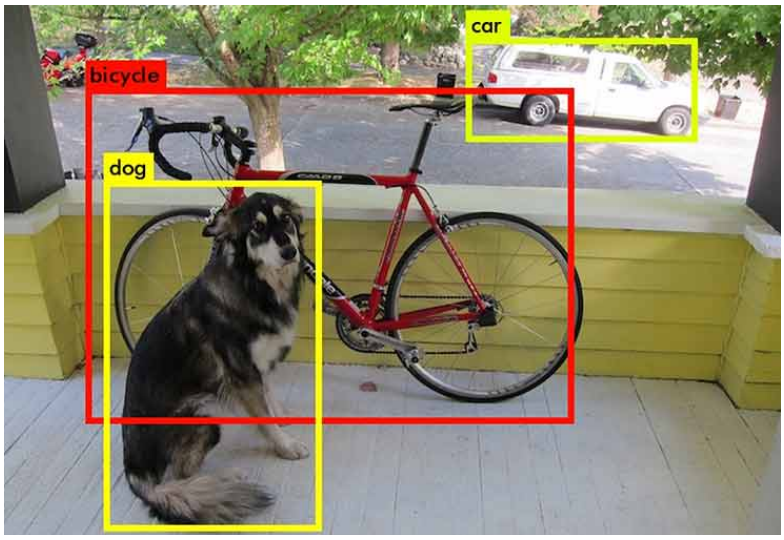


Figure 3.14.Object detection example with Deep Learning

3.3.1. Object Detection

Before discussing the technical details of object detection using deep learning, it is necessary to define object detection. The process of identifying objects using deep learning techniques focuses on recognizing the characteristics of the object to be identified in an image or video frame. These characteristics include what the object is (e.g., car, bicycle, or dog) as well as its dimensions, position, and size in the image plane (URL-6).

Deep learning object recognition is suitable for many applications in numerous fields,

including robotics, surveillance, and autonomous driving, because it is fast and accurate. For example, in autonomous driving vehicles, object detection is used to identify and avoid obstacles on the road, while in robotics it is used to identify and manipulate objects.

3.3.2. Deep Learning

Machine learning is a fundamental area within deep learning. Deep learning relies on the use of neural networks inspired by the structure and logic of the human brain, rather than machine learning itself. Figure 3.15 below simply illustrates the difference between deep and machine learning algorithms. It consists of interconnected layers of nodes where each node receives input from the previous layer and performs a mathematical operation before passing the output to the next layer (URL-7).

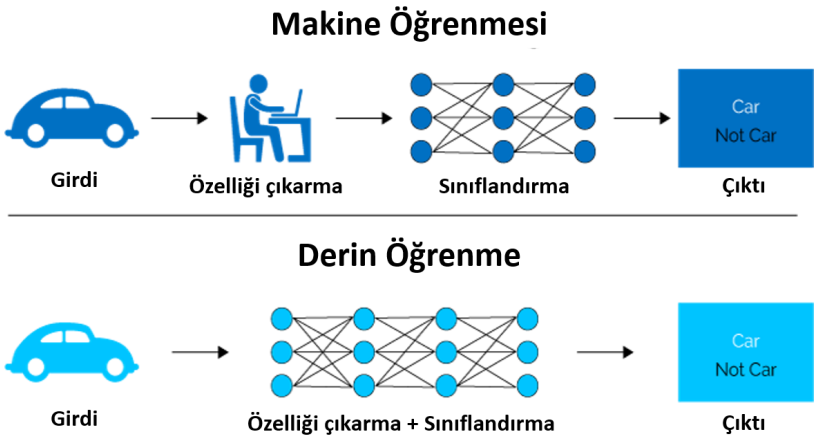


Figure 3.15.The difference between ML and DL.

Deep learning has revolutionized computer vision, enabling accurate and efficient object detection. It relies on recognizing patterns in a dataset and making predictions based on those patterns. In object detection, deep learning algorithms are trained on a large image dataset containing the objects of interest. The deep learning algorithm learns to identify objects while making predictions based on patterns in the data.

3.3.3. Object Detection Application with Deep Learning

After providing a basic overview of object detection and deep learning, the process of object detection using deep learning is discussed. This process consists of the following steps:

1. Collecting a dataset consisting of images
2. Adding an information note (label) to the dataset
3. Preprocessing the dataset
4. Training a deep learning model
5. Model evaluation
6. Using the model for object detection

3.3.3.1. Gathering a Dataset of Images

The first step in object detection with deep learning is to collect a dataset of images. The dataset should contain images that represent the objects you want to detect. For example, if you want to detect a glass, you should collect a dataset of images containing glasses.

It is important to collect various images representing the different angles, sizes, and lighting conditions from which the object might appear. This helps ensure the model can perceive the object in a variety of situations.

There are many publicly available datasets that can be used to train a model, such as COCO, ImageNet, and Pascal VOC. A custom dataset can also be created by capturing images or videos of the objects to be detected.

3.3.3.2. Adding a Label to the Dataset

After collecting the dataset of images, the next step is to add annotations to the dataset. This process involves labeling each image with information about the objects within the image. This information typically includes the object's position within the image and a label that identifies the object.

Several tools, such as Labeling and RectLabel, are available for adding annotations to datasets. These tools allow you to draw bounding boxes around objects in an image and label them with a descriptive tag. Adding annotations is a time-consuming process, but it is crucial for training a deep learning model.

3.3.3.3. Preprocessing the Data Set

After adding annotations to the dataset, the next step is preprocessing the images. At

this stage, the images from the training dataset undergo various modifications, such as reorienting, scaling from large to small or vice versa, flipping, and cropping. These modifications can be performed using techniques provided by the MATLAB program.

The process of modifying data is called data augmentation, and this is a crucial step because it helps avoid a phenomenon called overfitting in deep learning. Overfitting occurs when a model is taught so much about the data it has learned that it becomes highly specialized, which can lead to a decrease in the model's performance on new, unfamiliar data.

3.3.3.4. Training a Deep Learning Model

After adding information to the dataset and preprocessing it, the next step is to train the deep learning model. There are several deep learning programs that can be used for object detection, including TensorFlow, MATLAB, PyTorch, and Keras.

The training phase of a deep learning model focuses on crucial sub-phases aimed at reducing the resulting error rate. These include adjusting the model's weights at various training stages, feeding in the training data processed as described in the previous phase, and monitoring the error rate between the model's actual and expected outputs. This method is known as backpropagation in the field of deep learning.

3.3.3.5. Model Evaluation

At this stage, the deep learning model training is completed, and the process of measuring and evaluating the resulting model begins. This process is carried out by feeding evaluation data consisting of previously unused data (test dataset) to calculate the model's sensitivity and accuracy. Precision helps measure the model's accuracy in identifying the target, while accuracy helps measure the recognition rate.

3.3.3.6. Using the Model for Object Detection

After all the previous steps have been completed and the desired results obtained, the Model is ready to be used for object detection. Object detection involves importing an image into the model and using the output to draw bounding boxes around the objects in the image. Several object detection algorithms are available for use with deep learning,

such as YOLO, SSD, and Faster R-CNN. These algorithms differ in speed and accuracy, and the choice of algorithm depends on the application.

4. CONCEPTUAL DEFINITIONS IN ROBOTICS

A robot moves in three-dimensional space in relation to the objects around it. For this, the relative positions and orientations of both the robot and the surrounding objects must be defined. A coordinate system must be assigned to each object in three-dimensional space, including the robot, to determine its position and orientation. These coordinate systems, placed on the objects, must be related to a master coordinate system where the positions and orientations are fixed. All positions and orientations to be defined will be subjected to mathematical operations both with respect to the master coordinate system and to other coordinate systems within it. For simplicity, the "Cartesian coordinate system" is referred to as CCS (Cartesian Coordinate System) throughout this thesis.

4.1. Location Description

By defining a CCS (Correctional System), a point in space can be positioned anywhere within the main CCS. Generally, more than one CCS can be placed within the main CCS. A point in 3D space can be represented by a 3x1 vector relative to the center of this CCS. These vectors are named according to the CCS in which they are defined. For example, the position of a point in the main CCS relative to the CCS is represented by a vector. The mathematical expression is shown below;$P\{A\}\,{}^{A}_{\square}P_{\square}\,{}^{A}_{\square}P_{\square}$

$$ {}^{A}_{\square}P_{\square}=\begin{bmatrix} {}^{A}_{\square}P_{x} \\ {}^{A}_{\square}P_{y} \\ {}^{A}_{\square}P_{z} \end{bmatrix} \tag{4.1} $$

As previously stated, CCSs are placed on the robot and the objects around it. The vector between the center of and the center of is shown as follows;$\{A\}\{B\}\,{}^{A}_{\square}P_{B}\,{}^{A}_{\square}P_{B}$

$$ {}^{A}_{\square}P_B = \begin{bmatrix} {}^{A}_{\square}P_{B_x} \\ {}^{A}_{\square}P_{B_y} \\ {}^{A}_{\square}P_{B_z} \end{bmatrix} \tag{4.2} $$

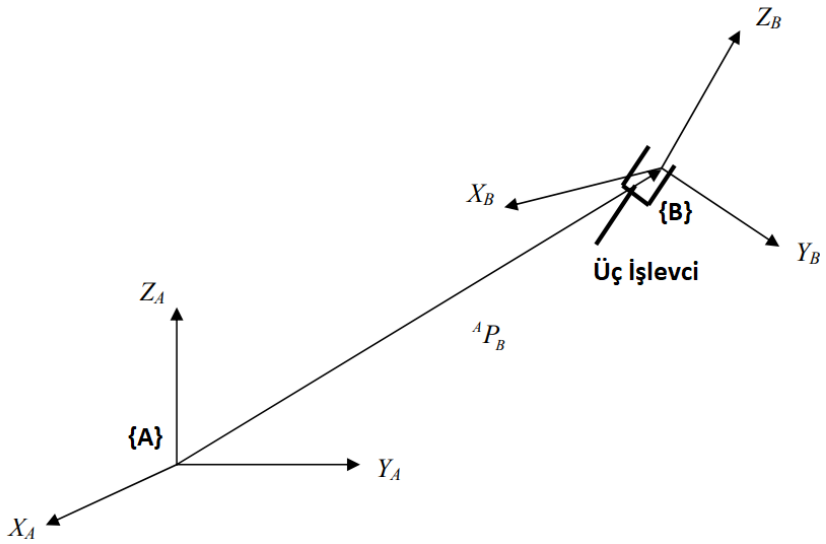


Figure 4.1. Vector representation between and$\{A\}\{B\}$

4.2. Orientation Definition

The orientation of a CCS placed on an object in 3D space is defined in addition to the object's position relative to any other CCS. Orientation expresses the amount of rotation (turning) of one CCS relative to another and is represented by a matrix.$3\,x\,3$

To define the orientation of a rigid body relative to another reference CCS, a CCS must be placed on the rigid body. As shown in Figure 4.1, the orientation relative to the reference CCS is defined by placing three functionals on the rigid body. Unit vectors are used to define the orientation in terms of the reference CCS, which are placed on the three functionals. Let's express the unit vectors of the CCSs as in (4.3) and (4.4);$\{B\}$ $\{A\}\{B\}\{A\}\{A\}\{B\}$

$$ \{A\} = {}^{\square}_{\square}\hat{X}_A, {}^{\square}_{\square}\hat{Y}_A, {}^{\square}_{\square}\hat{Z}_A \tag{4.3} $$

$$ \{B\} = {}^{\square}_{\square}\hat{X}_B, {}^{\square}_{\square}\hat{Y}_B, {}^{\square}_{\square}\hat{Z}_B \tag{4.4} $$

$\{B\}$The unit vectors of can be expressed as follows:$\{A\}$

$^{A}_{\square}\hat{X}_{B}$ $^{A}_{\square}\hat{Y}_{B}$ $^{A}_{\square}\hat{Z}_{B}$

This expression can be represented by a matrix. The newly created matrix is called the rotation matrix. Since this matrix defines the orientation of with respect to , it is shown as ;$3\,x\,3\{B\}\{A\}^{A}_{B}R$

$$^{A}_{B}R=\left[^{A}_{\square}\hat{X}_{B}\quad ^{A}_{\square}\hat{Y}_{B}\quad ^{A}_{\square}\hat{Z}_{B}\right]=\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \tag{4.5}$$

Each column of the matrix in (4.5) contains a unit vector. These unit vectors define the orientation of the axes of with respect to . Let the centers of and be at the same point, as in Figure 4.2. Here, the orientation of the unit vector in with respect to is shown as and is defined as follows (Bingöl et al., 2005);$\{B\}\{A\}\{B\}\{A\}^{\square}_{\square}\hat{X}_{B}\{B\}\{A\}^{A}_{\square}\hat{X}_{B}$

$$^{A}_{\square}\hat{X}_{B}=\begin{bmatrix} r_{11} \\ r_{21} \\ r_{31} \end{bmatrix}=\begin{bmatrix} X_{B} \vee X_{A} \vee \cos\gamma_{AB} \\ X_{B} \vee Y_{A} \vee \cos\beta_{AB} \\ X_{B} \vee Z_{A} \vee \cos\alpha_{AB} \end{bmatrix}=\begin{bmatrix} ^{\square}_{\square}\hat{X}_{B}\,^{\square}_{\square}\hat{X}_{A} \\ ^{\square}_{\square}\hat{X}_{B}\,^{\square}_{\square}\hat{Y}_{A} \\ ^{\square}_{\square}\hat{X}_{B}\,^{\square}_{\square}\hat{Z}_{A} \end{bmatrix} \tag{4.6}$$

Using this equation, the matrix can be rewritten as follows:$^{A}_{B}R$

$$^{A}_{B}R=\begin{bmatrix} ^{\square}_{\square}\hat{X}_{B}\cdot\,^{\square}_{\square}\hat{X}_{A} & ^{\square}_{\square}\hat{Y}_{B}\cdot\,^{\square}_{\square}\hat{X}_{A} & ^{\square}_{\square}\hat{Z}_{B}\cdot\,^{\square}_{\square}\hat{X}_{A} \\ ^{\square}_{\square}\hat{X}_{B}\cdot\,^{\square}_{\square}\hat{Y}_{A} & ^{\square}_{\square}\hat{Y}_{B}\cdot\,^{\square}_{\square}\hat{Y}_{A} & ^{\square}_{\square}\hat{Z}_{B}\cdot\,^{\square}_{\square}\hat{Y}_{A} \\ ^{\square}_{\square}\hat{X}_{B}\cdot\,^{\square}_{\square}\hat{Z}_{A} & ^{\square}_{\square}\hat{Y}_{B}\cdot\,^{\square}_{\square}\hat{Z}_{A} & ^{\square}_{\square}\hat{Z}_{B}\cdot\,^{\square}_{\square}\hat{Z}_{A} \end{bmatrix} \tag{4.7}$$

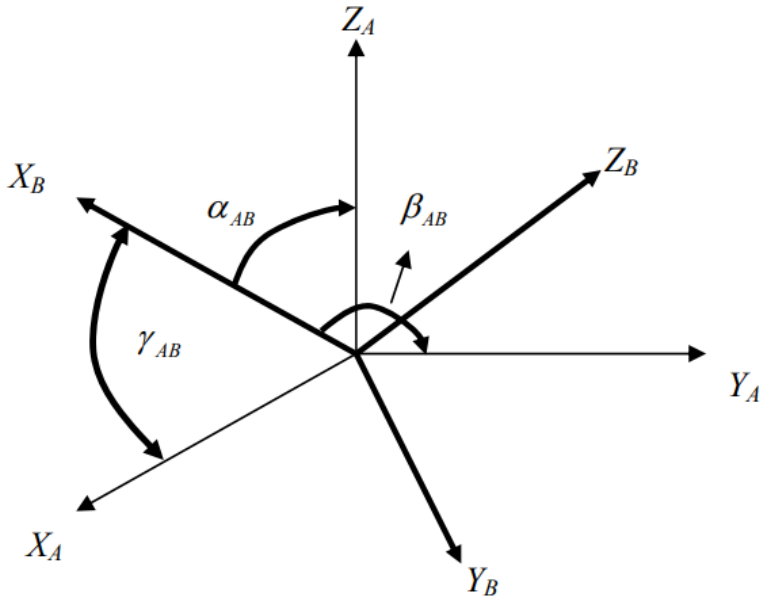


Figure 4.2.Two CCSs with the same center but different orientations.

The equation can be expressed more clearly when rearranged;

$$ {}^{A}_{B}R = \begin{bmatrix} X_B \vee X_A \vee \cos\gamma_{AB} & Y_B \vee X_A \vee \cos\gamma_{AB} & Z_B \vee X_A \vee \cos\gamma_{AB} \\ X_B \vee Y_A \vee \cos\beta_{AB} & Y_B \vee Y_A \vee \cos\beta_{AB} & Z_B \vee Y_A \vee \cos\beta_{AB} \\ X_B \vee Z_A \vee \cos\alpha_{AB} & Y_B \vee Z_A \vee \cos\alpha_{AB} & Z_B \vee Z_A \vee \cos\alpha_{AB} \end{bmatrix} \tag{4.8} $$

The rotation matrix of an orthogonal matrix can be defined as follows:

$$ R = \begin{bmatrix} \hat{X} & \hat{Y} & \hat{Z} \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \tag{4.9} $$

Since the matrix R is an orthogonal matrix, the columns of the matrix satisfy the following equations;

$\hat{X} \vee 1, |\hat{Y} \vee 1 \quad \hat{Z} \vee 1$

$\hat{X}.\hat{Y}=0, \qquad \hat{X}.\hat{Z}=0 \qquad \hat{Y}.\hat{Z}=0$

$\hat{X}.\hat{X}=1, \qquad \hat{Y}.\hat{Y}=1 \qquad \hat{Z}.\hat{Z}=1$

$\begin{bmatrix} A \end{bmatrix}$The orientation of 'in' with respect to 'is shown by the matrix as shown in equation (4.10);$\begin{bmatrix} B \end{bmatrix}{}^{A}_{B}R$

$$ {}^{B}_{A}R=\left[{}^{B}_{\square}\hat{X}_{A}\quad {}^{B}_{\square}\hat{Y}_{A}\quad {}^{B}_{\square}\hat{Z}_{A}\right]=\begin{bmatrix} {}_{\square}\hat{X}_{A}\,{}_{\square}\hat{X}_{B} & {}_{\square}\hat{Y}_{A}\,{}_{\square}\hat{X}_{B} & {}_{\square}\hat{Z}_{A}\,{}_{\square}\hat{X}_{B} \\ {}_{\square}\hat{X}_{A}\,{}_{\square}\hat{Y}_{B} & {}_{\square}\hat{Y}_{A}\,{}_{\square}\hat{Y}_{B} & {}_{\square}\hat{Z}_{A}\,{}_{\square}\hat{Y}_{B} \\ {}_{\square}\hat{X}_{A}\,{}_{\square}\hat{Z}_{B} & {}_{\square}\hat{Y}_{A}\,{}_{\square}\hat{Z}_{B} & {}_{\square}\hat{Z}_{A}\,{}_{\square}\hat{Z}_{B} \end{bmatrix} \tag{4.10} $$

$\{A\}$The rotation matrix defining 'yi' with respect to 'yi' is equal to the transpose of its inverse matrix (Bingöl et al., 2005).$\{B\}{}^{B}_{A}R$

${}^{B}_{A}R$Let the matrix be its inverse. Multiplying it by gives the identity matrix;${}^{B}_{A}R^{-1}{}^{B}_{A}R{}^{B}_{A}R^{-1}$

$$ {}^{B}_{A}R\,{}^{B}_{A}R^{-1}=I \tag{4.11} $$

$\{A\}$If the centers of and coincide, a point defined with respect to can be defined with respect to and can be shown mathematically as in equation (4.12);$\{B\}\{B\}{}^{B}_{\square}P\{A\}$

$$ {}^{A}_{\square}P={}^{A}_{B}R\,{}^{B}_{\square}P \tag{4.12} $$

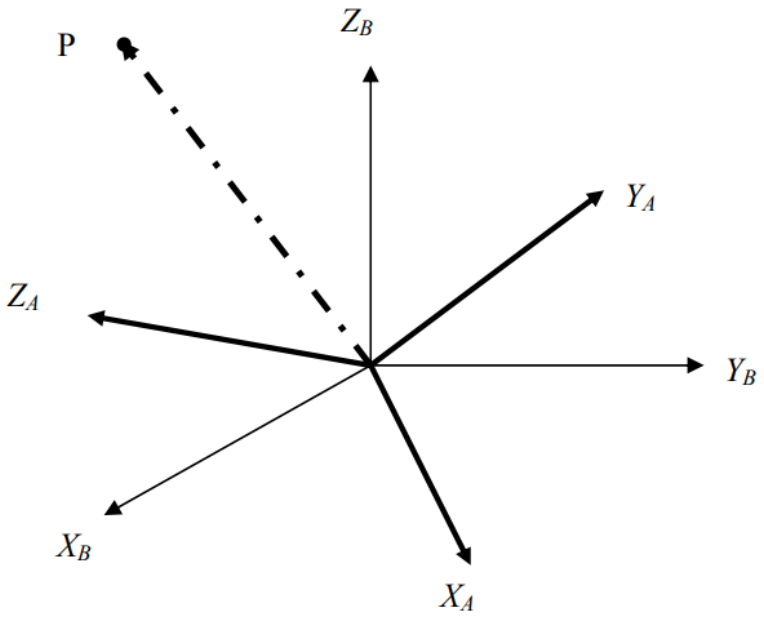


Figure 4.3.Representation of two concentric CCSs.

$\{A\}$Let us assume that there are two CCSs, and , but let the position of with respect to be denoted by the vector. Figure 4.4 shows two CCSs with the same orientation but different centers at different points. The position of point with respect to can be defined by the vector as in equation (4.13);$\{B\}\{A\}\{B\}{}^{A}_{\square}P_{B}P\{A\}{}^{A}_{\square}P_{B}$

$$ {}^{A}_{\square}P = {}^{B}_{\square}P + {}^{A}_{\square}P_{B} \tag{4.13} $$

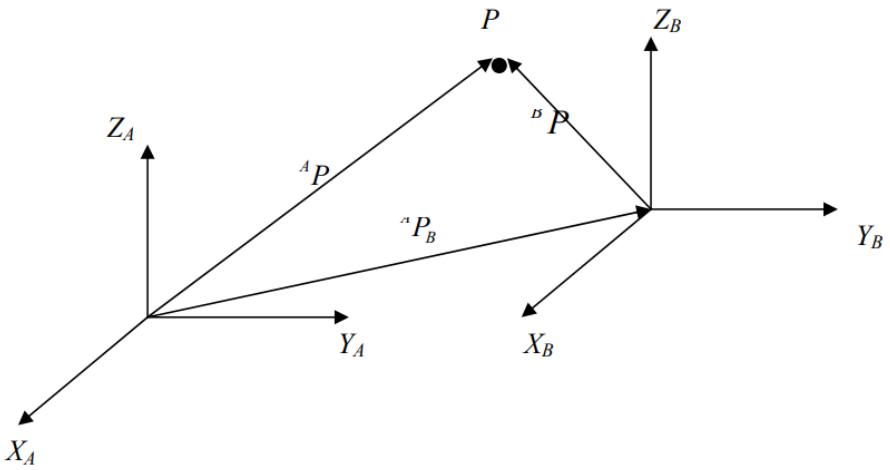


Figure 4.4.Graphical representation of equation (4.13)

Let's take it a step further and assume there are two CCSs with different orientations and centers at different points, as shown in Figure 4.5. The position of point ' with respect to ' is shown as in equation (4.14);$P\{A\}$

$$ {}^{A}_{\square}P = {}^{A}_{\square}P_{B} + {}^{A}_{B}R\,{}^{B}_{\square}P \tag{4.14} $$

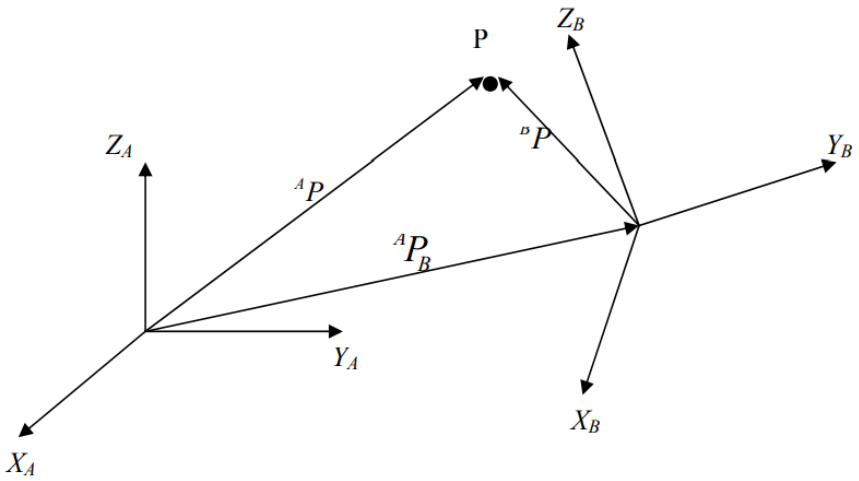


Figure 4.5.Two CCSs with different orientations and centers in different locations.

If the orientations of and are the same but their centers are different (Figure 4.4), it is shown as an equation and satisfies equation (4.13). If the centers of and are at the same point but have different orientations (Figure 4.3), it is shown as an equation and satisfies equation (4.12).$\{A\}\{B\}{}^{A}_{\square}P = {}^{B}_{\square}P + {}^{A}_{\square}P_{B}\{A\}\{B\}{}^{A}_{\square}P = {}^{A}_{B}R\,{}^{B}_{\square}P$

Generally, to describe its relationship with a single matrix, it is defined by the transformation matrix as shown in equation (4.15);$^{B}_{\square}P\,^{A}_{\square}P\,^{A}_{B}T$

$$^{A}_{\square}P = {}^{A}_{B}T\,{}^{B}_{\square}P \tag{4.15}$$

Equation (4.15) can be written in detail as follows;

$$\begin{bmatrix} {}^{A}_{\square}P \\ 1 \end{bmatrix} = \begin{bmatrix} {}^{A}_{B}R & {}^{A}_{\square}P_{B} \\ 0_{1x3} & 1 \end{bmatrix} \begin{bmatrix} {}^{B}_{\square}P \\ 1 \end{bmatrix} \tag{4.16}$$

4.3. Homogeneous Transformation Matrix

$^{A}_{\square}P$The relationship between and can be expressed by a matrix containing the rotation matrix and the position vector. This matrix is called the homogeneous transformation matrix and is shown as follows (Bingöl et al., 2005);$^{B}_{\square}P\,4\,x\,4$

$$^{A}_{B}T = \begin{bmatrix} {}^{A}_{B}R & {}^{A}_{\square}P_{B} \\ 0_{1x3} & 1 \end{bmatrix} \tag{4.17}$$

$^{A}_{B}T$Let's denote the matrix as its inverse. While finding the matrix using known methods is quite difficult and requires lengthy operations, finding the inverse of orthogonal matrices is as simple as taking its transpose. Let's try to find it by taking advantage of the orthogonality property as shown in equation (4.18);$^{A}_{B}T^{-1}\,^{A}_{B}T^{-1}\,^{A}_{B}T^{-1}$

$$^{A}_{B}T^{-1} = {}^{B}_{A}T = \begin{bmatrix} {}^{B}_{A}R & {}^{B}_{\square}P_{A} \\ 0_{1x3} & 1 \end{bmatrix} \tag{4.18}$$

It is known that the inverse of the matrix in the equation is equal to its transpose. However, the procedure for is slightly different. To obtain the matrix, the position vector in the equation must be written as a function of the matrix.$^{B}_{A}R\,R^{-1} = {}^{A}_{B}\square\,{}^{B}_{A}R = {}^{A}_{B}R^{T}$

$^{B}_{\square}P_{AB}\,^{A}T^{-1}\,^{B}_{\square}P_{AB}\,^{A}R$

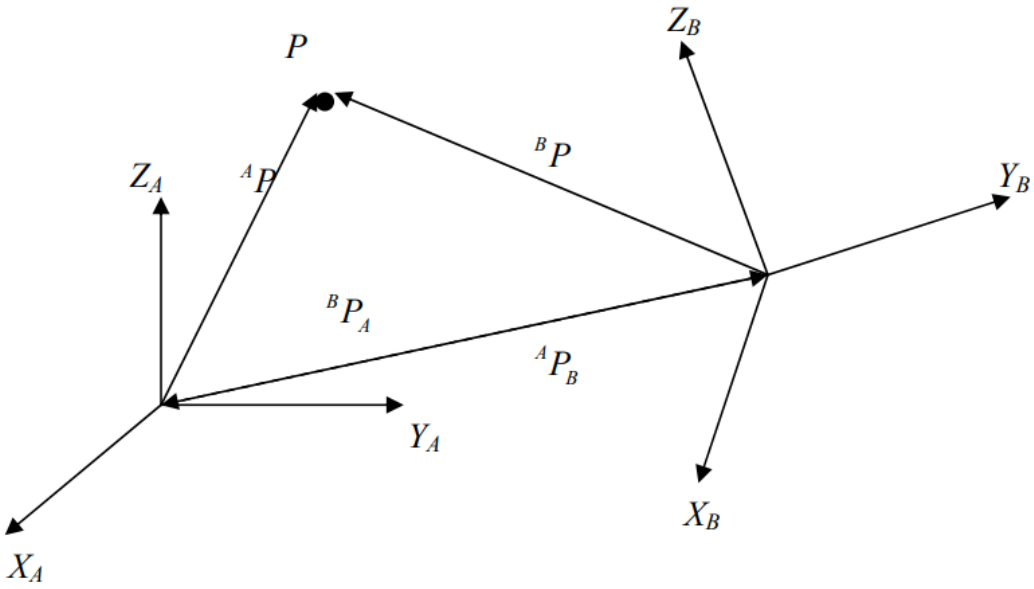


Figure 4.6. definition of point in terms of different CCSsP

This is obtained by following the method described in (Bingöl et al., 2005). Thus, the position vector expression is obtained. ${}^{B}_{\square}P^{\square}_{A} = -{}^{B}_{A}R\,{}^{A}_{\square}P^{\square}_{B}\,{}^{B}_{\square}P^{\square}_{A}\,{}^{B}_{\square}P^{\square}_{A} = -{}^{A}_{B}R^{T}_{\square}\,{}^{B}_{\square}P^{\square}_{A}$

When the found expressions are substituted into the form, it is found as follows: ${}^{B}_{A}R\,{}^{B}_{\square}P^{\square}_{A}$

$$ {}^{A}_{B}T^{-1} = \begin{bmatrix} {}^{B}_{A}R & {}^{B}_{\square}P^{\square}_{A} \\ 0_{1x3} & 1 \end{bmatrix} \quad {}^{A}_{B}T^{-1} = \begin{bmatrix} {}^{A}_{B}R^{T}_{\square} & -{}^{A}_{B}R^{T}_{\square}\,{}^{A}_{\square}P^{\square}_{B} \\ 0_{1x3} & 1 \end{bmatrix} $$

To obtain the inverse of a transformation matrix, first the transpose of the rotation matrix is calculated. The resulting matrix is written into the rotation part of the transformation matrix. Then, the transposed rotation matrix is multiplied by the position vector, and a negative sign is placed before the new expression. This newly formed expression is also written into the position vector part of the transformation matrix, thus completing the process of inverting a transformation matrix.

4.4. Advanced Kinematics of Robotics

Forward kinematics deals with the position, velocity, and acceleration information of robot joints. If a coordinate system is placed on the robot joints, the relationship between two adjacent joints is expressed by a transformation matrix. The matrix is the transformation matrix that connects the robot's fixed reference platform to the first joint. The last matrix connects the robot's last joint to three functionors. The relationship

between the CSS of the three functionors and the reference CSS of the fixed platform is determined using successive transformation matrices for each joint of the robot, as shown in equation (4.19). This relationship is called forward kinematics and expresses the position and orientation of the gripper relative to the fixed platform (Bingöl et al., 2005);${}_{i}^{i-1}T\,{}_{1}^{0}T$

$$ {}_{N}^{0}T = {}_{1}^{0}T\,{}_{2}^{1}T\,{}_{3}^{2}T \ldots {}_{N}^{N-1}T \tag{4.19} $$

Figure 4.7 provides a schematic representation of the forward kinematics problem of a planar robot (URL-8).

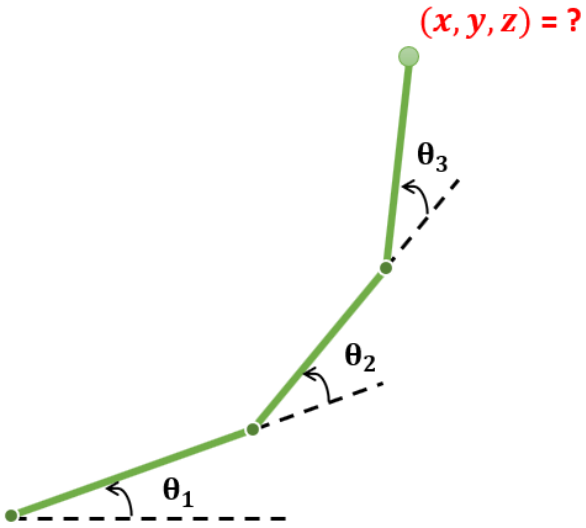


Figure 4.7. Advanced kinematic representation.

4.5. Inverse Kinematics of Robots

Inverse kinematics is one of the fundamental stages of robot control. According to the main CCS, inverse kinematics is the process of finding the joint variables of a robot based on the position and orientation information of three given functions. Figure 4.8 provides a schematic representation of the inverse kinematics problem of a planar robot (URL-8). The figure shows the angle that the robot arms must take for the coordinates of the end gripper.

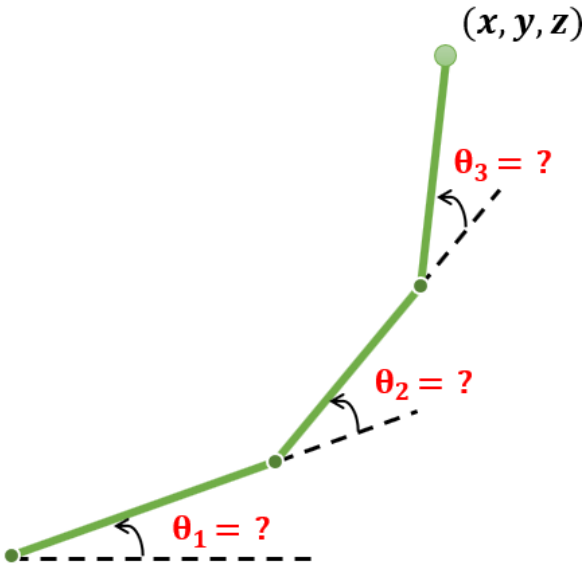


Figure 4.8. Inverse kinematic representation.

5. KINEMATIC MODEL OF THE DELTA ROBOT

The primary purpose of a kinematic model is to calculate the joint angles given the position of the end actuator and vice versa. The process of calculating the position coordinates from the joint angles is called forward or manipulator kinematics. Inverse kinematic analysis, on the other hand, is the calculation of joint angles given the desired position coordinates of the end actuator (Doğan, 2010) (Williams, 2016). Figure 5.1 shows the schematic diagram of the Delta robot. Table 5.1 specifies the parameters of the Delta robot used.

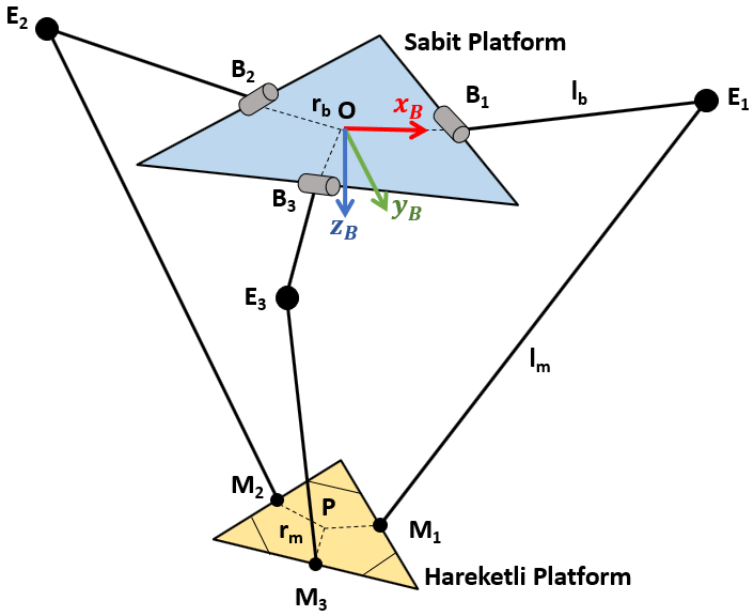


Figure 5.1.Delta Parallel Robot

Table 5.1.Parameters of the Delta robot used.

Parameter	Explanation	Value (mm)
lb	Upper arm length	250
lm	lower arm length	1070
rb	Fixed platform radius	208
rm	Moving platform radius	64

As shown in Figure 5.1, the parallel delta robot is fixed to the ceiling with an equilateral triangle-shaped platform. The coordinate system assigned to this platform (upper platform) is placed at the center point O and is considered the "Base frame" or "Zero frame". The robot also consists of a movable platform (lower platform). The center of gravity of this platform is considered as the P (x, y, z) operation point or the end

function of this parallel robot. The platforms that make up this robot are connected to each other by means of three arms. These arms can bend inwards and outwards thanks to the spherical joints (E1, E2, E3) located in the middle. The rotary joints (B1, B2, B3) and spherical joints (M1, M2, M3) located in the middle of each edge of the fixed platform and the movable platform, respectively, provide the connection of the platform to the ends of the arms. In Figure 5.2, the assignment of the coordinate systems is made and the relevant symbols are given. The necessary frameworks have been assigned in a way that minimizes computation; the correct assignment of coordinate systems and the selection of symbols are crucial for better resolution and understanding of the model.

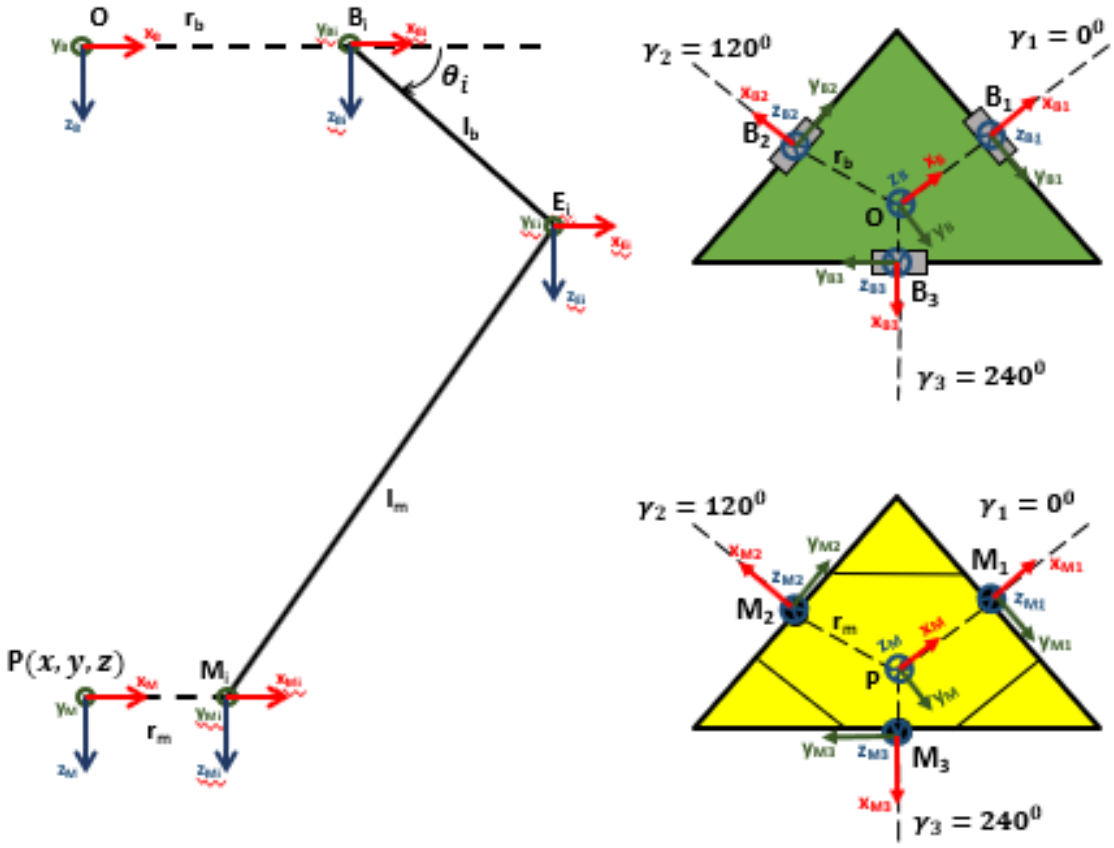


Figure 5.2. Assignment of coordinate systems

A summary of the coordinate system assignment is given in Tables 5.2, 5.3, and 5.4.

Table 5.2. Fixed platform coordinate systems

Coordinate system	Original	Coordinate axes
{HE}	HE	xB, yB, zB
{B1}	B1	xB1, yB1, zB1
{B2}	B2	xB2, yB2, zB2

{B3}	B3	xB3, yB3, zB3

Table 5.3. Coordinate systems of the middle joints of the arms

Coordinate system	Original	Coordinate axes
{E1}	E1	xE1, yE1, zE1
{E2}	E2	xE2, yE2, zE2
{E3}	E3	xE3, yE3, zE3

Table 5.4. Mobile platform coordinate systems

Coordinate system	Original	Coordinate axes
{P}	P	xM, yM, zM
{M1}	M1	xM1, yM1, zM1
{M2}	M2	xM2, yM2, zM2
{M3}	M3	xM3, yM3, zM3

Based on the definitions in Figure 5.2, the following transformation matrices exist between coordinate systems.

The transformation matrix between the coordinate systems {O} and {Ei}; ${}^{KS_O}_{KS_{Ei}}T$

$$ {}^{KS_O}_{KS_{Ei}}T = \begin{bmatrix} \cos\gamma_i & -\sin\gamma_i & 0 & \cos\gamma_i l_b \cos\theta_i + r_b \cos\gamma_i \\ \sin\gamma_i & \cos\gamma_i & 0 & \sin\gamma_i l_b \cos\theta_i + r_b \sin\gamma_i \\ 0 & 0 & 1 & l_b \sin\theta_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{5.1} $$

The transformation matrix between the coordinate systems {O} and {M}; ${}^{KS_O}_{KS_M}T$

$$ {}^{KS_O}_{KS_M}T = \begin{bmatrix} \cos\gamma_i & -\sin\gamma_i & 0 & r_m \cos\gamma_i + x \\ \sin\gamma_i & \cos\gamma_i & 0 & r_m \sin\gamma_i + y \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{5.2} $$

The transformation matrix between the coordinate systems {O} and {P}; ${}^{KS_O}_{KS_P}T$

$$ {}^{KS_O}_{KS_P}T = \begin{bmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & y \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{5.3} $$

It has been taken as follows here. $\gamma_i = 0^0, 120^0, 240^0$

As shown in Figure 5.2, since the distance between points E and M is equal to , the magnitude of the vector is;l_m $\overrightarrow{EM}$

$$\left\|\overrightarrow{EM}\right\|^2 = l_m^2 \tag{5.4}$$

equals.

Using the position vectors of the transformation matrices in equations (5.1), (5.2) and (5.3) and based on the vector equation, the coordinates of the vector are; $\overrightarrow{EM} = \overrightarrow{OM} - \overrightarrow{OE}$ $\overrightarrow{EM}$

$$\overrightarrow{EM} = \begin{bmatrix} x - \cos\gamma_i l_b \cos\theta_i - r_b \cos\gamma_i \\ y - \sin\gamma_i l_b \cos\theta_i - r_b \sin\gamma_i \\ z - l_b \sin\theta_i \end{bmatrix}$$

As shown above, by using the vector distance formula, when the coordinate values of the vector are substituted into equation (5.4), the resulting equation will be as in (5.5); $\overrightarrow{EM}$

$$\left(x - \cos\gamma_i l_b \cos\theta_i - r_b \cos\gamma_i\right)^2 + \left(y - \sin\gamma_i l_b \cos\theta_i - r_b \sin\gamma_i\right)^2 + \left(z - l_b \sin\theta_i\right)^2 = l_m^2 \tag{5.5}$$

Equation (5.5) is then simplified as follows:

$$\left(x - x_i\right)^2 + \left(y - y_i\right)^2 + \left(z - z_i\right)^2 = l_m^2 \tag{5.6}$$

It is obtained here;

$$x_i = \left(r + l_b \cos\theta_i\right)\cos\gamma_i \qquad y_i = \left(r + l_b \cos\theta_i\right)\sin\gamma_i \qquad z_i = l_b \sin\theta_i$$

It is possible.

5.1. DPR Advanced Kinematic Calculations

In advanced kinematic calculations, the position coordinates of the end effector are solved using the joint angles of the manipulator; in parallel robots, the end effector is located at the center of gravity of the movable platform.

When the value in equation (5.6) is replaced, the following three equations are obtained;

i

$$(x-x_1)^2+(y-y_1)^2+(z-z_1)^2=l_m^2 \tag{5.7}$$

$$(x-x_2)^2+(y-y_2)^2+(z-z_2)^2=l_m^2 \tag{5.8}$$

$$(x-x_3)^2+(y-y_3)^2+(z-z_3)^2=l_m^2 \tag{5.9}$$

Using equations (5.7) and (5.8), the solution in terms of the value of is:xz

$$x=\frac{a_1}{d_1}z+\frac{b_1}{d_1} \tag{5.10}$$

is obtained. Here;

$$a_1=(z_3-z_2)(y_2-y_1)-(z_2-z_1)(y_3-y_2)$$

$$b_1=\left(\frac{w_2-w_1}{2}\right)(y_3-y_2)-\left(\frac{w_3-w_2}{2}\right)(y_2-y_1)$$

$$d_1=(x_2-x_1)(y_3-y_2)-(x_3-x_2)(y_2-y_1)$$

It has been taken as such.

By following the same steps, equations (5.8) and (5.9) are used to obtain the solution of in terms of the value of , where is;$z\,y\,y$

$$y=\frac{a_2}{d_2}z+\frac{b_2}{d_2} \tag{5.11}$$

is obtained. Here, however;

$$a_2=(z_3-z_2)(x_2-x_1)-(z_2-z_1)(x_3-x_2)$$

$$b_2=\left(\frac{w_2-w_1}{2}\right)(x_3-x_2)-\left(\frac{w_3-w_2}{2}\right)(x_2-x_1)$$

$$d_2=(x_3-x_2)(y_2-y_1)-(x_2-x_1)(y_3-y_2)$$

It has been taken as such.

Finally, by substituting the values found in equations (5.10) and (5.11) into equation (5.6), a quadratic equation with one unknown is obtained;$x\,y\,z$

$$\left[\frac{a_1^2}{d_1^2}+\frac{a_2^2}{d_2^2}+1\right]z^2+\left[\frac{2a_1b_1}{d_1^2}+\frac{2a_2b_2}{d_2^2}+\frac{2a_1x_1}{d_1}-\frac{2a_2y_1}{d_2}-2\right]z+\left[\frac{b_1^2}{d_1^2}+\frac{b_2^2}{d_2^2}+\frac{2b_1x_1}{d_1}-\frac{2b_1y_1}{d_2}-l_b^2-w\right.$$

The value is found by solving this second-degree equation. The found value is substituted into equations (5.10) and (5.11) and their values are calculated. Thus, the position of the point is found according to the base coordinate system.$z\,z\,x\,y\,P(x,y,z)$

5.2. DPR Inverse Kinematic Calculations

To find the DPR angle values, equation (5.5) was expanded to obtain expression (5.12); $\sin\theta_i$

$$\sin\theta_i=\frac{\psi_i-(x\cos\gamma_i+y\sin\gamma_i-r)\cos\theta_i}{z} \tag{5.12}$$

Here ;ψ_i

$$\psi_i=\frac{l_b{}^2-l_m{}^2-(x-r\cos\gamma_i)^2+(y-r\sin\gamma_i)^2+z^2}{2l_b}$$

It is defined as such.

$\cos^2\theta_i+\sin^2\theta_i=1$By substituting the formula into equation (5.12), an expression in the form of a second-degree equation with only one unknown is obtained;$\sin\theta_i$

$$a\cos^2\theta_i+b\cos\theta_i+c=0 \tag{5.13}$$

The equations a, b, and c used are as follows:

$$a = z^2 + \left(x\cos\gamma_i + y\sin\gamma_i - r\right)^2$$

$$b = -2\psi_i\left(x\cos\gamma_i + y\sin\gamma_i - r\right)$$

$$c = \psi_i^{\,2} - z^2$$

It is found by solving the equation given in (5.13).$\cos\theta_i = \frac{-b+\sqrt{\Delta}}{2a}$

Then, the values are found by solving equation (5.13) and substituting the values found in the formula.$\theta_i\theta_i = atan2\left(\sin\theta_i, \cos\theta_i\right)\cos\theta_i$

5.3. Numerical Simulation and Validation of Kinematic Analysis

To verify the accuracy of the forward and inverse kinematic results obtained in this section, a circular trajectory with a diameter of 300 mm was first used to obtain the inverse kinematics. Then, the TK solution was used to verify the position information of the end function using the obtained HR solutions. Figure 5.3 shows the verification of the end function's position by comparing it with the reference trajectory using the obtained forward kinematics. The positioning error obtained using the TK and HR is shown in Figure 5.4.

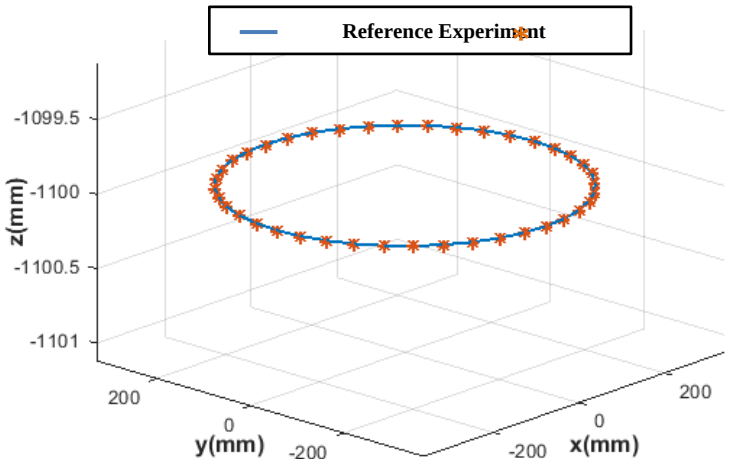


Figure 5.3.Forward and reverse kinematic verification graph

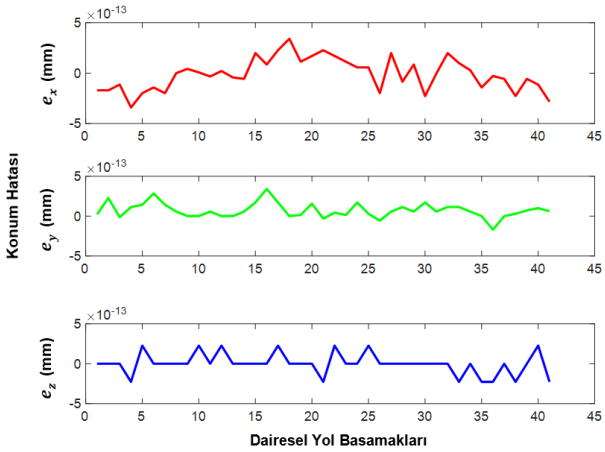


Figure 5.4.X, Y and Z Position Error

In this study, a MATLAB program was used to determine all the points that the delta robot can reach, in other words, the robot's working area (Smirnova, 2023). The parameters of the delta robot used are specified in Table 5.1. The mechanical limitations of the real-world system application were taken into account. The results of the 3D working area obtained using the resulting HR and TK equations are shown in Figures 5.5, 5.6 and 5.7.

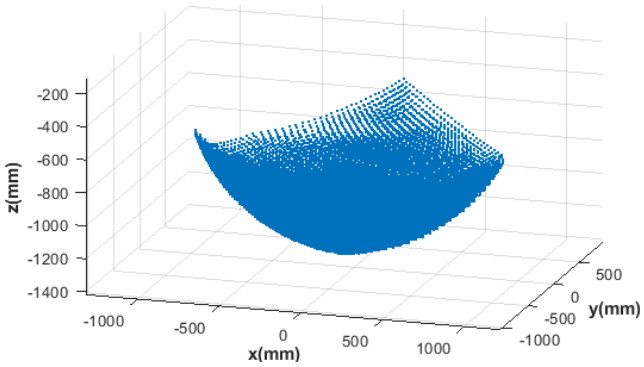


Figure 5.5.3D view of the Delta workspace.

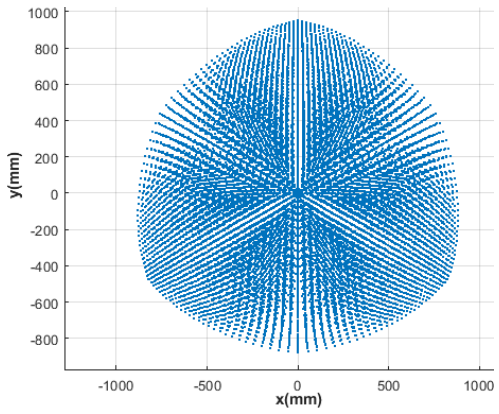


Figure 5.6.Top view of the Delta workspace.

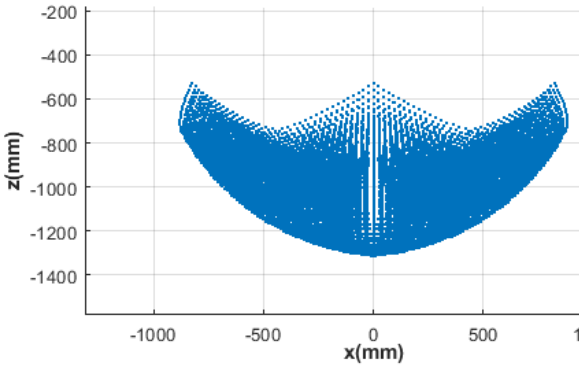


Figure 5.7.Front view of the Delta workspace.

6. SYSTEM DESIGN

This section of the study discusses the design of the working system, which is divided into two main systems: the robotic control system and the computer vision system.

As shown in Figure 6.1, the delta robot has a 3-DOF parallel mechanism and consists of three limbs. Each limb has one rotary active joint and two spherical passive joints. In the implementation of this system, the delta robot is used with three AC servo motors and drivers. The primary purpose of the servo motors is to provide mobility in the active joints of the manipulator. A conveyor belt is used for material transfer. A Logitech C310 HD webcam is mounted on the conveyor belt along which the objects move. The delta robot picks up the required objects (PET bottles) and places them in the sorting area according to the computer vision system. Unnecessary objects are shown as wooden cubes.

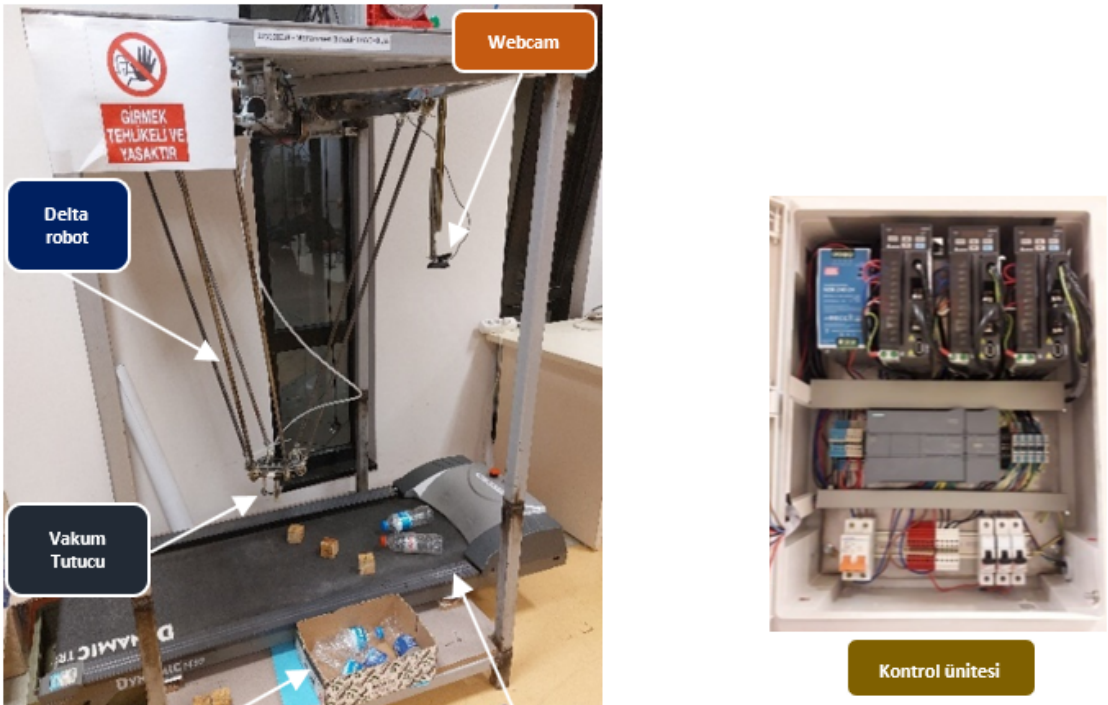


Figure 6.1.Designed robotic system and control panel

As shown in Figure 6.2, the computer vision system consists of a computer and a webcam. The motion control system consists of a computer, PLC, servo drives, and servo motors. Communication between the computer vision system and the robotic

control system is carried out via the OPC protocol.

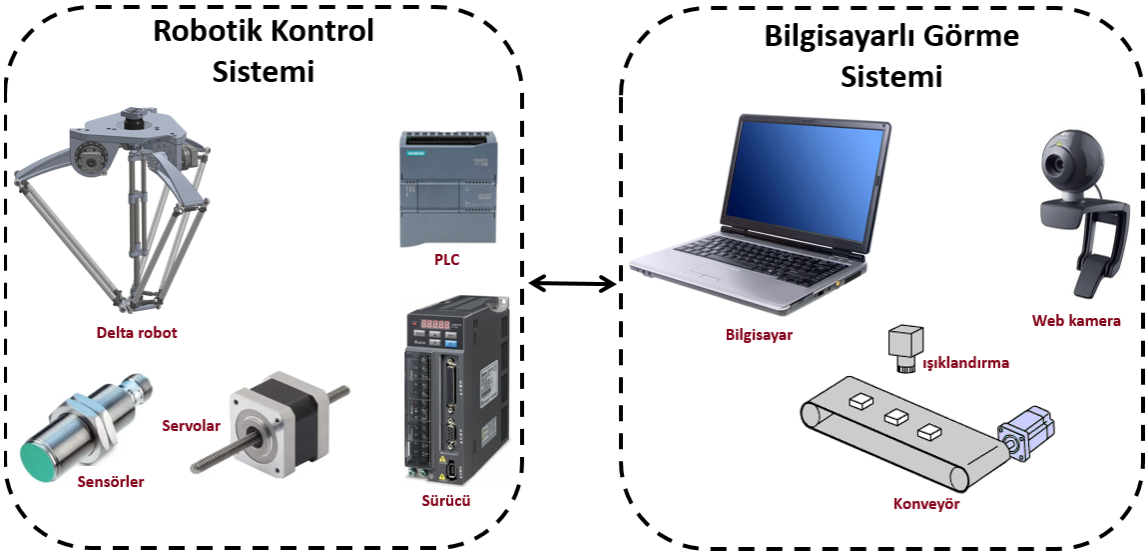


Figure 6.2.Components of the implemented system

The computer vision system is responsible for acquiring information about objects for the system's motion control. In this study, a webcam was selected as the input device for imaging purposes. The webcam used is a digital color device with a resolution of 1280 × 720 pixels. The vertical distance between the webcam and the conveyor was determined as 700 mm with a 60° diagonal field of view. The computer vision system consists of four main modules: a visual processing module that uses deep learning and image processing algorithms for detection; a visual positioning module to determine the location of detected objects and calculate their position coordinates; an aggregate position calculation module to predict the target point and update position information; and a motion planning module to calculate angle values using kinematic model results. This system communicates with the robotic control system. It calculates the absolute displacement and enables the servo motors to move the delta robot to sort the acquired objects.

The robotic control system is built on a Siemens S7-1215C PLC. Motion control function blocks are programmed to control the servomotors. Ladder logic programming language was used to write the PLC code. This programming language was deemed suitable for this study due to its efficiency and ease of debugging.

7. PLC-based servo motor position control.U

The PLC S7-1200 offers the option of using pre-configured commands to control servo motors and servo drives. The ease of configuring and subsequently controlling the axis truly makes this technology worth considering. Axis positioning can be done very quickly, making it well-suited for many applications.

Motion control can be achieved in three different ways: by directly influencing the drive software via pulse sequence output (PTO), analog output, or PROFINET connection. In this study, motion control was performed using the PTO output.

7.1. Setting up servo motor control with PLC

Servo motors are motors developed for easy control of position, speed, and torque. They are more precise motors than asynchronous motors, which are commonly used in automation systems. They consist of a motor, a feedback module (such as an encoder), and a control circuit.

Implementing PLC-based servo motor control is a crucial aspect of industrial automation and requires attention to detail and systematic execution. Initially, it is necessary to assemble the required hardware components, including a PLC compatible with servo motor control and a corresponding servo drive. These components form the basic structure of the control system, facilitating precise motion control.

Figure 7.1 shows the modules and pseudo-connections that control the motion of the movable end function, the vacuum suction cup, in the designed robotic system. Figure 7.2 shows the electrical connection diagram between the PLC and the servo motor driver. Since the driver can be connected as both PNP and NPN, the PLC's output connection type is important here. Because the S7-1200 PLC has a PNP output, a PNP connection was preferred for the servo driver (URL-9). Therefore, the common terminal (COM) of the driver is connected as negative (-).

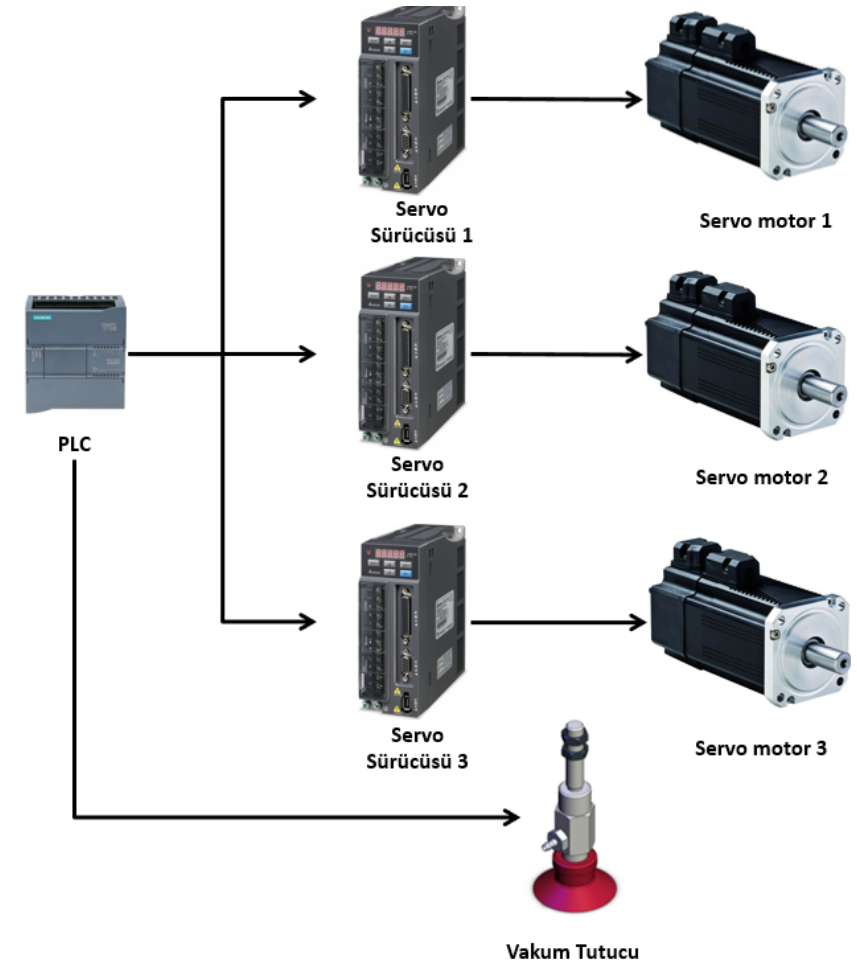


Figure 7.1.Equipment used in robotic systems

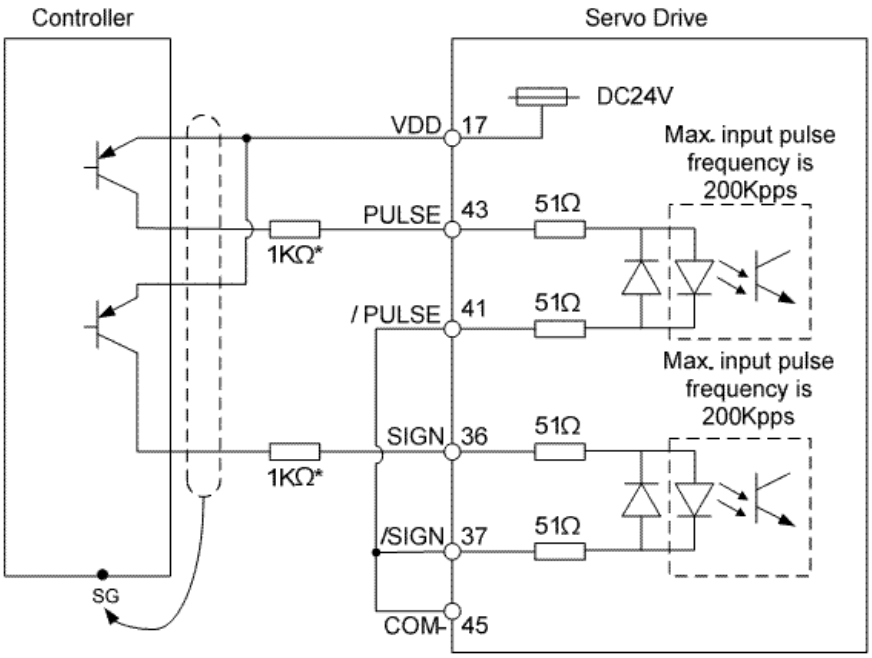


Figure 7.2.PLC controller and servo drive PNP cable connection.

After the hardware connections are made as shown in Figure 7.2, the next step is to establish communication between the PLC and the servo drive. This is accomplished using communication protocols such as EtherCAT or Modbus, which enable seamless data exchange between the two devices.

Once communication is successfully established, the final step involves configuring both the PLC and servo drive parameters according to the desired motion and control requirements. This includes settings such as speed, acceleration, deceleration, and positioning parameters. Precise adjustment of these parameters ensures optimum performance and accuracy in servo motor control, enabling it to meet the specific requirements of the application.

7.2. PLC and Servo Configuration in TIA Portal

The following steps were followed to configure the servo motor and PLC in the TIA portal program (URL-10):

First, the PLC is added to the project (see Figure 7.3).

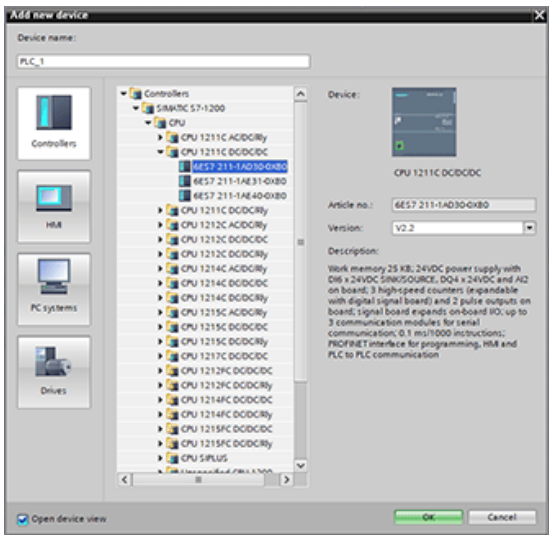


Figure 7.3.Adding a PLC device to the project

After addition, the first pulse generators, PTO1/PWM1, PTO2/PWM2, and PTO3/PWM3, can be enabled by navigating to the Device Configuration tab (see Figure 7.4).

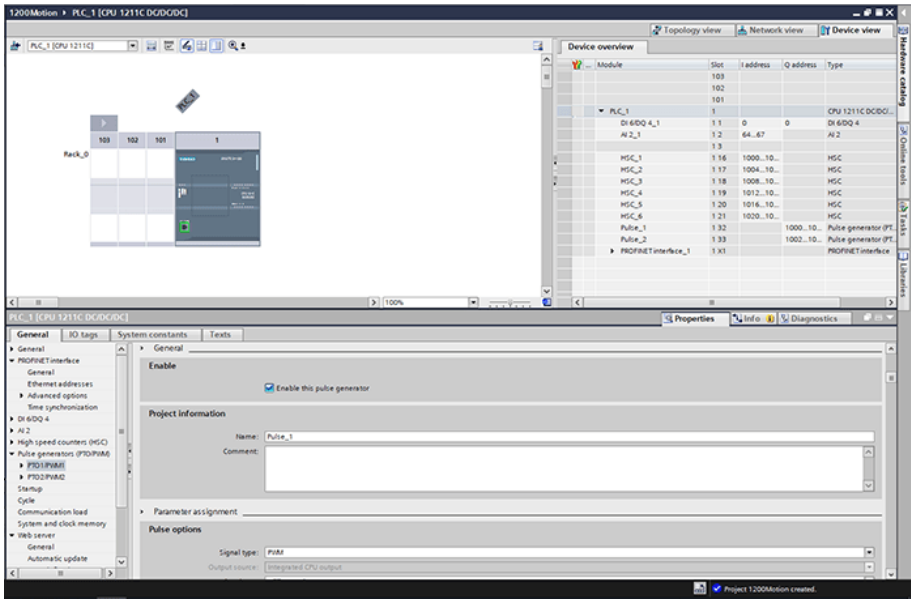


Figure 7.4.PLC Device Configuration

Since the servo drives will be driven by pulse control, the PTO signal type of the pulse generator has been activated in the project (see Figure 7.5).

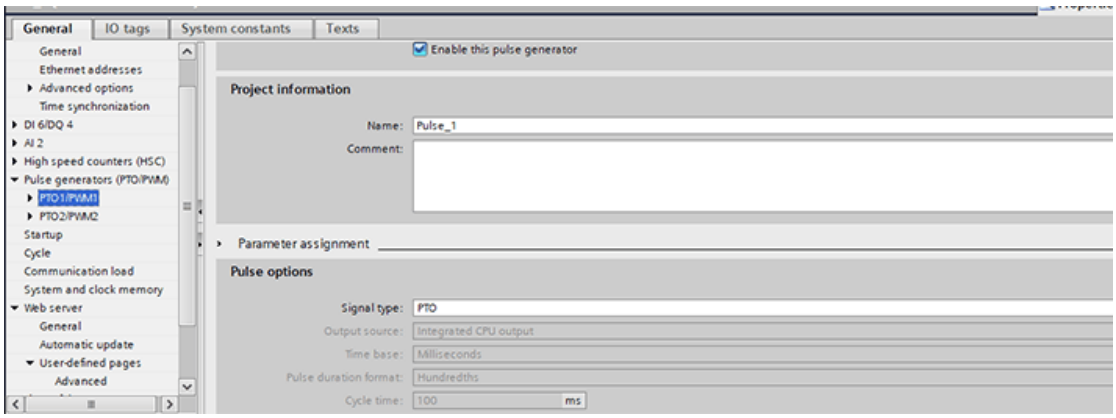


Figure 7.5.Selecting the PTO signal

The pulse output was selected as Q0.0, the direction output as Q0.3, and the enable output as Q0.6. A summary of the output assignments for the 3 servos used in this study is given in Table 7.1.

Table 7.1.Pulse output, Direction output, and Activation output pin assignments.

	Pulse output	Direction exit	Activation output
Servo 1	Q0.0	Q0.3	Q0.6
Servo 2	Q0.1	Q0.4	Q0.7
Servo 3	Q0.2	Q0.5	Q1.0

In the next step, an axis is created from the technology object. Click "Add new object"

under the Technology Objects folder and select "TO_AXIS_PTO" (see Figure 7.6).

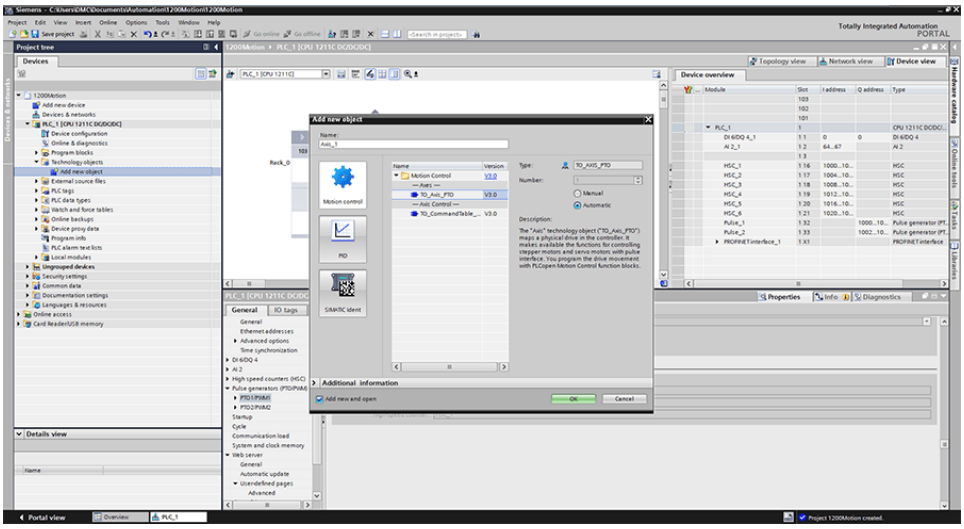


Figure 7.6.Adding an Axis PTO technology object.

Next, when the technology object is created, “Pulse_1” should be selected, and its unit should be degrees (see Figure 7.7).

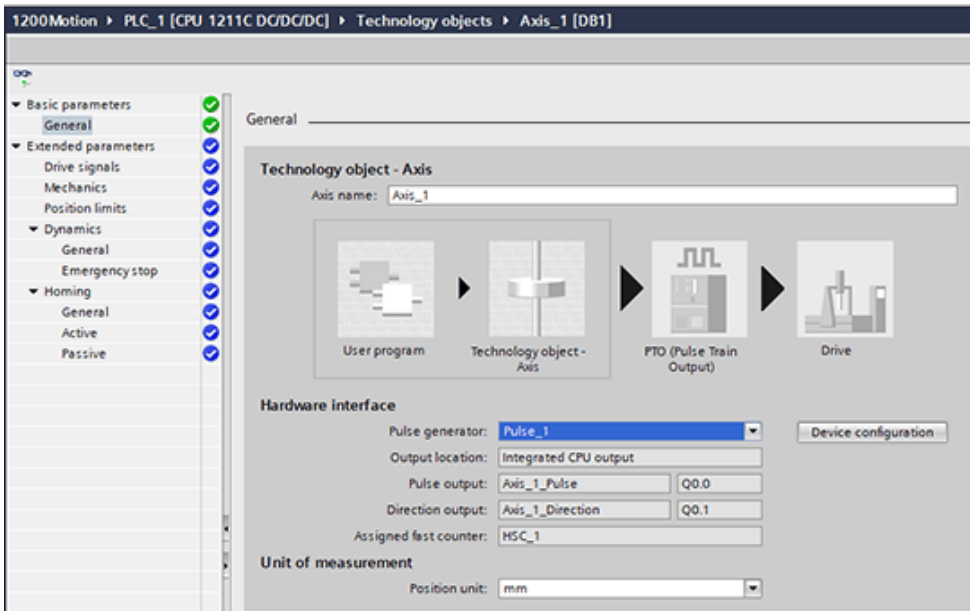


Figure 7.7.Axis PTO technology object configuration

Finally, the expanded parameters select pulses per motor revolution and load movement per motor revolution. This ensures that the speeds and positions given by the command written in the program are accurate (see Figure 7.8).

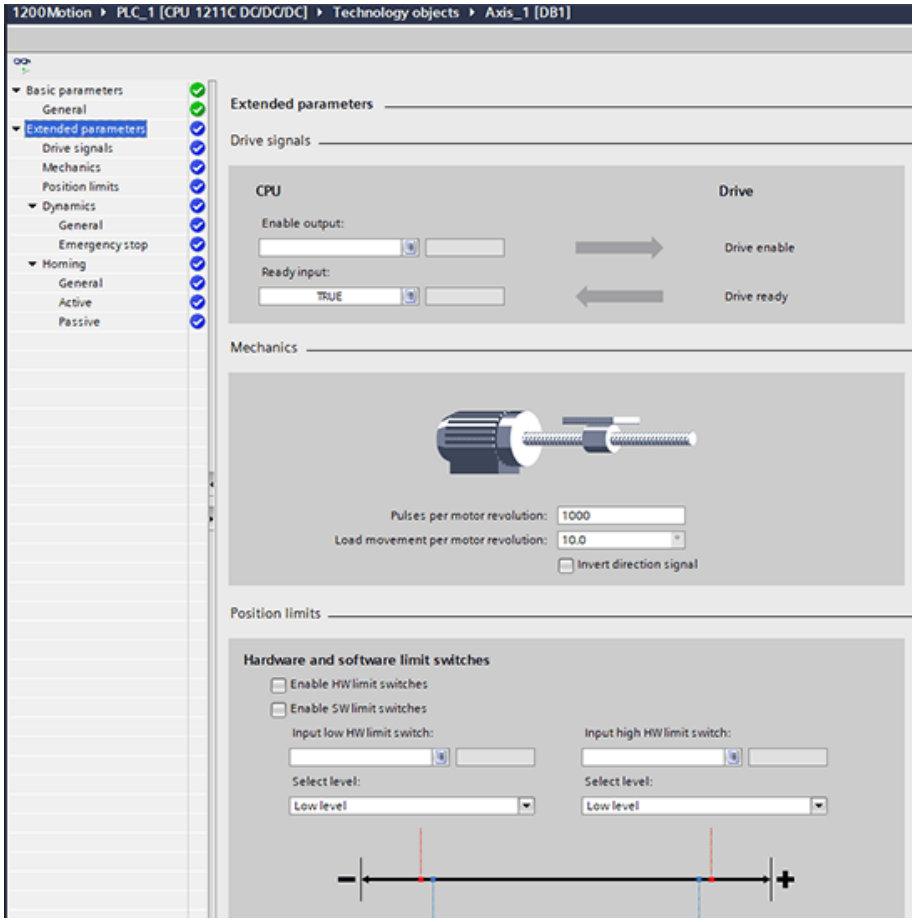


Figure 7.8.adjustment of mechanical and dynamic parameters

Once the technology object is configured, the servo motor is ready for programming. However, before that, it needs to be tested to see if Axis is working. For this purpose, the commissioning tool is used within the technology object.

First, the axis configuration was loaded into the PLC. Then, the Axis created for the servo drive was selected from the project tree, and then the Commissioning (1) tab, which provides a panel for manual control of the drive, was opened. After establishing a connection with the PLC, the drive was activated (2). Finally, the axis (3) was enabled, and it was observed that the motor went to the values entered in the interface. In this way, control over the drive was achieved, and it was seen that the drive was correctly configured with the technology object in the project (see Figure 7.9).

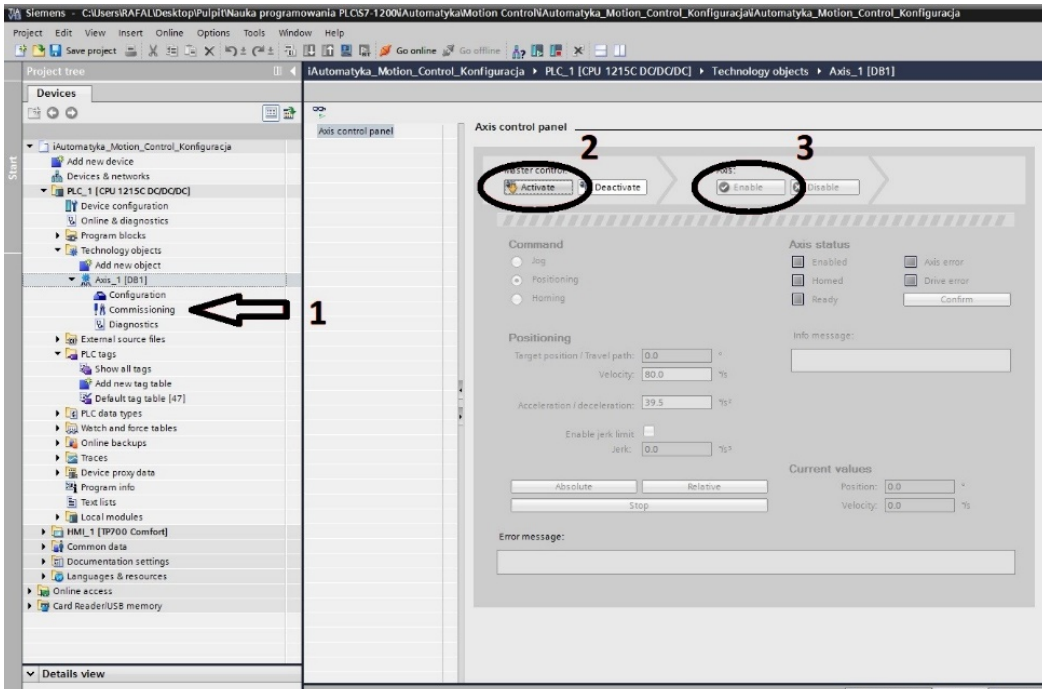


Figure 7.9.Axis Commissioning process

This study uses three motion command blocks. Before going into the details of these commands, it is necessary to draw attention to their status outputs. These status outputs provide valuable information regarding the execution of the motion command. This information is given below:

- Done: Mission successfully completed.
- Busy: The task is currently being processed,
- CommandAborted: The mission was terminated by another job during execution.
- Error: An error occurred while processing the task.
- ErrorId: Displays the cause of the error in hexadecimal format.
- In programming the Axis, the MC_POWER command can be used to activate it, MC_HOME to home it, and MC_MOVEABSOLUTE to position it.

The motion control commands used in the PLC code for this project are described below:

MC_POWER Command: Servos do not operate without this command block. Data blocks that define the movement coordinates are connected to the “Axis” inputs. For the drive connected to the Axis input of the MC_POWER block shown in Figure 7.10 to be

active, the block's “Enable” input must be active.

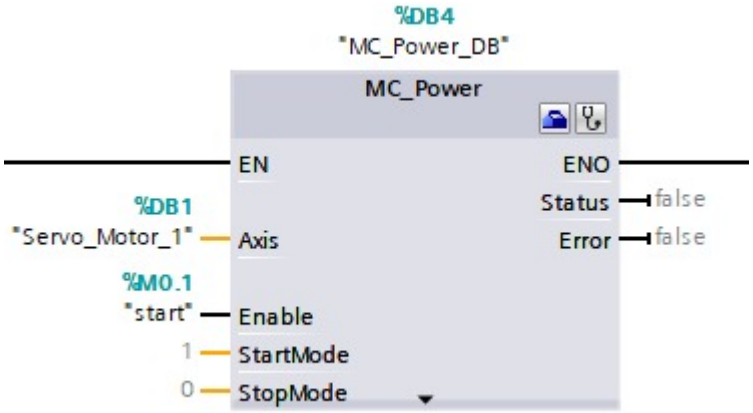


Figure 7.10.MC_POWER command block

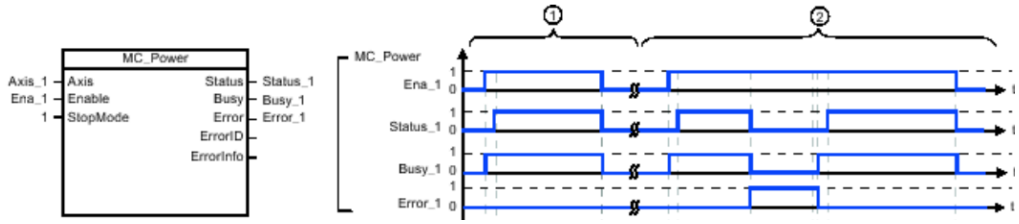


Figure 7.11.The graph of the MC_POWER instruction block.

As shown in Figure 7.11 (URL-10):

①: An axis is enabled and then disabled again (as shown in Ena_1), “Busy_1” is high. Successful activation can be read via “Status_1”.

2: As shown in Error_1 in the figure above, the axis was disabled due to an error that occurred after the activation process. The axis will resume operation from where it left off once the error is resolved.

MC_HOME Command: Referencing a servo axis is the process of specifying a known position or position for the axis of a servo motor with reference to the mechanical setup of your system. This process is typically performed at the start of operation of a motion control system or after a system has been shut down or restarted. Referencing is a crucial part of setting up and configuring your motion control system, as it ensures the correct operation of the system and the accurate and reliable movement of the servo

motor axis. In TIA Portal, a servo axis can be referenced using the MC_HOME command (see Figure 7.12).

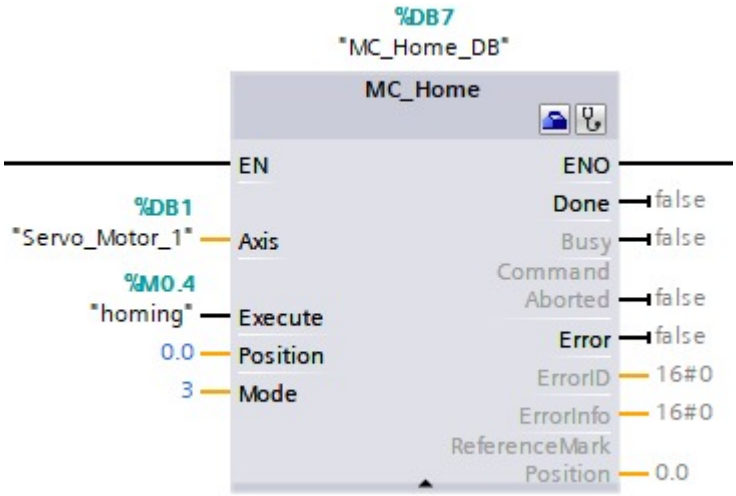


Figure 7.12.MC_HOME command block

When using the MC_HOME Motion Control command to reference a servo axis, the Mode input can be set as follows:

- **0**Passive homing.
- **1**Active homing.
- **2**Adjusting the actual position.
- **3**: Relative offset from the actual position.
- **4**Absolute encoder position offset.

The appropriate homing mode for a servo axis depends on the encoder type. These encoders are either incremental encoders (single turn) or absolute encoders (multiple turns). An axis with an incremental encoder does not maintain its position after a power cycle. The axis must be homed after each power cycle. A servo axis with an absolute encoder maintains its position after a power cycle, so there is no need to home it after each power cycle.

MC_MOVEABSOLUTE: This command block moves the axis to an absolute position. For example, if the servo axis is at the 25 mm position and the command is issued with the Position input set to 50 mm, the axis will move to the 50 mm position and stop there (see Figure 7.13).

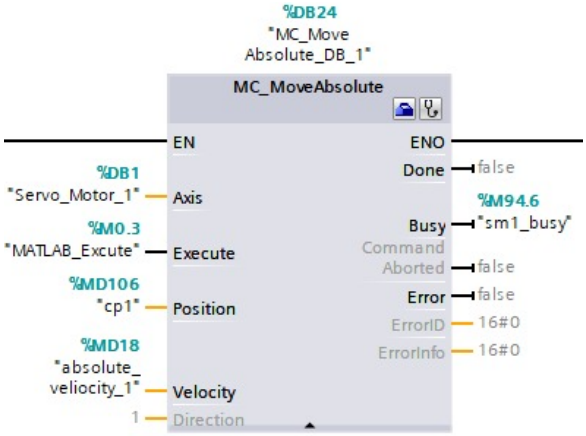


Figure 7.13.MC_MOVEABSOLUTE command block

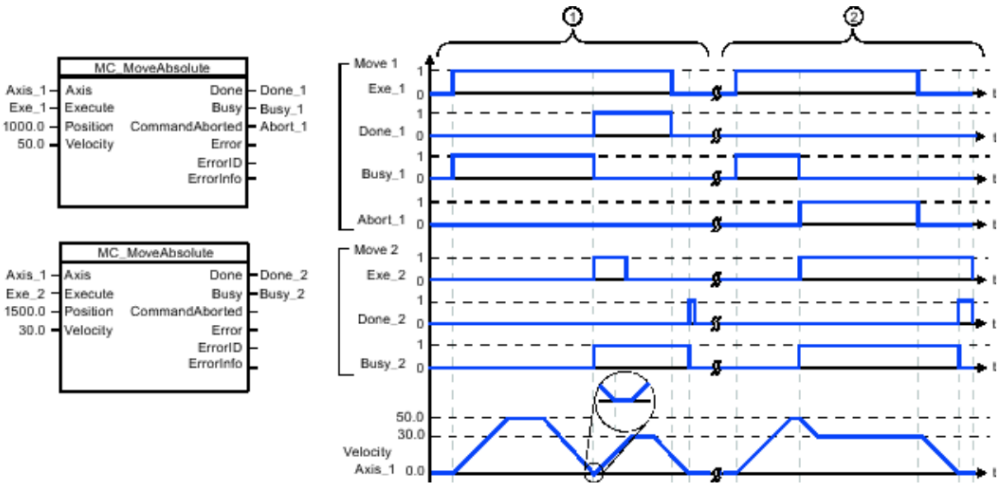


Figure 7.14.The graph of the MC_MOVEABSOLUTE command block.

In Figure 7.14 (URL-10) as seen:

①: Axis_1 is moved to the absolute position 1000.0 with a velocity of 50.0 using an MC_MoveAbsolute task. “Done_1” is activated to indicate that the axis has reached the target. The MC_MoveAbsolute process restarts from the last position reached when “Done_1” = TRUE, moving to the target position 1500.0. “Done_2” is activated to indicate that the axis has reached the new target.

2The active MC_MoveAbsolute operation is canceled when another operation starts. This cancellation is indicated by “Abort_1”. The axis moves the newly activated MC_MoveAbsolute operation to the new position 1500.0 at a new speed. “Done_2” is activated when the target is reached.

7.3. Control Program Created in PLC

The Delta robot was controlled using an S7-1200 PLC. This PLC was programmed using Ladder Logic in TIA Portal. The networks involved in the program code are described below:

In the network shown in Figure 7.15, MC_POWER blocks have been added and StartMode has been set to “1”. When the Enable input is TRUE, all three servo axes are activated.

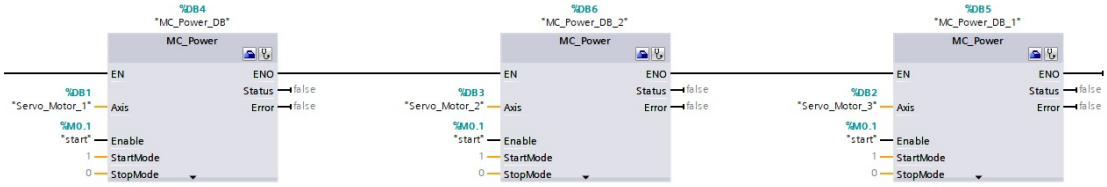


Figure 7.15.Powering the servo axes.

The next network, as shown in Figure 7.16, has MC_home blocks added for homing. In this study, the Mode input is set to “3” (as described in the MC_Home command section).

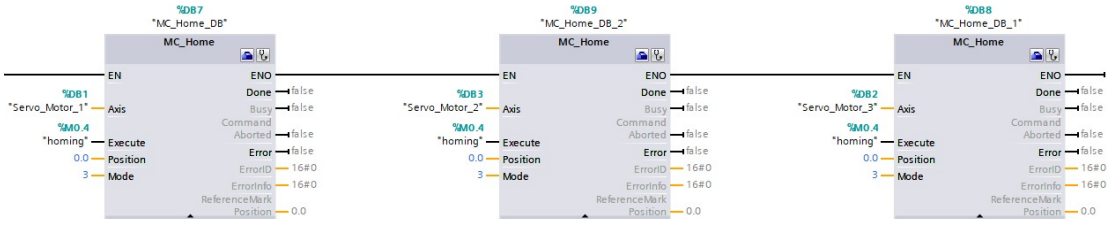


Figure 7.16.Home Servo Axes

MC_MOVEABSOLUTE blocks have been added to the network described in Figure 7.17, where the Excute input is linked to feedback data from the computer vision system in MATLAB.

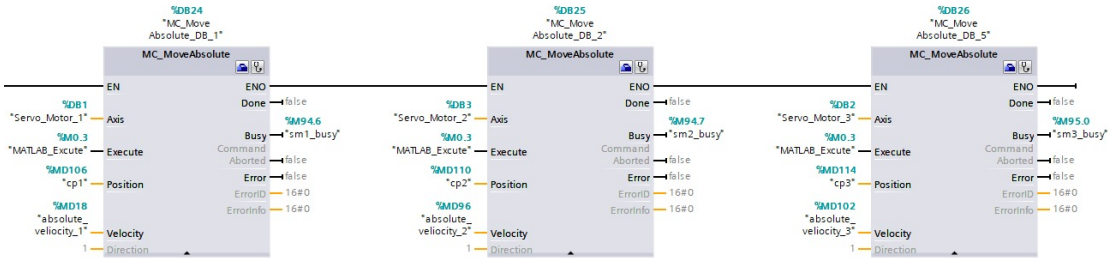


Figure 7.17.Moving the servo axes.

8. CAMERA CALIBRATION FOR VISUAL POSITIONING PURPOSES

Camera calibration is a crucial process for computer vision and plays a vital role in 3D reconstruction and image position analysis. During the camera calibration process, image distortion is corrected, and both intrinsic and extrinsic parameters are calculated to ensure accurate measurements and reliable analysis.

In this study, camera calibration was performed to obtain the K, R, and t matrices representing the camera parameters. K is the matrix representing the camera's intrinsic parameters, while R and t are the matrices representing the camera's extrinsic parameters. These matrices are used to find the 3D coordinates of a point from its corresponding 2D coordinates. In this study, camera calibration was performed programmatically using the camera calibration functions provided by MATLAB; the steps followed in this process are explained in the following sections.

8.1. Preparation of Calibration Images

Typically, the calibration pattern is a symmetrical checkerboard design containing a series of both black and white squares. In this study, the calibration pattern was printed and placed on the conveyor belt. The pattern used has a checkerboard size of 25 mm and is printed on an A3 (297x420) sheet of paper. Sample images used in the calibration process are shown in Figure 8.1.

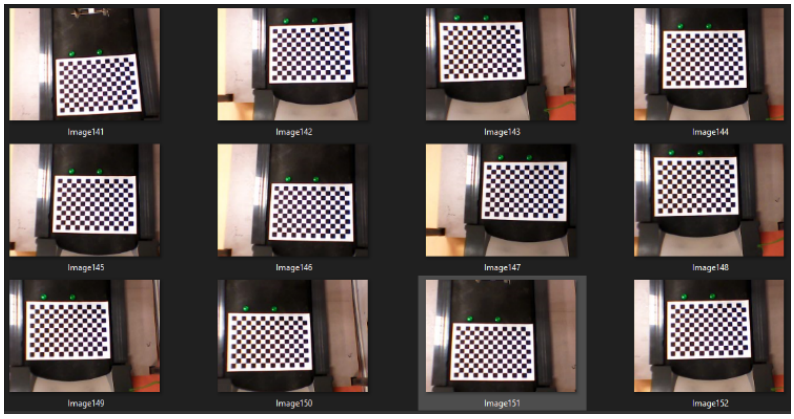


Figure 8.1.Sample images used in calibration

As shown in Figure 8.1, images of the calibration pattern used were taken from different angles. Different angles are necessary to obtain the focal length and distortion parameters.

8.2. Camera Calibration

After the image acquisition phase was completed, the calibration program code was run to determine the camera parameters. Before determining these parameters, it was necessary to identify the checkerboard corner points (image points) in the image, as shown in Figure 8.2. Then, the coordinates of the vertices' ground points in the pattern-centered coordinate system had to be generated. These image points and ground points were then used to calculate the camera parameters.

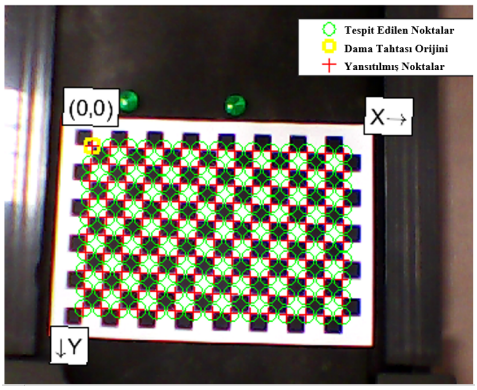


Figure 8.2.Calibration process

8.3. Evaluation of Calibration Accuracy

The bar graph shown in Figure 8.3 illustrates the accuracy of the calibration. Each bar represents the average reprojection error for the respective calibration image. A total of 8 images were used in the calibration process. The reprojection errors shown in the figure below represent the distance between the ideal Earth points and the detected points projected onto the image plane.

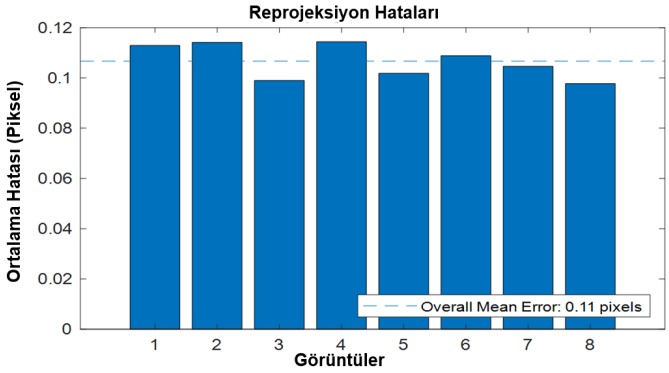


Figure 8.3.Projection error graph

Calibration accuracy can be improved based on the overall average error of the resulting

reprojection errors. However, due to the low quality of the camera used, some of the images taken for calibration purposes were deleted due to high reprojection errors and replaced with others to obtain better results.

8.4. Parameter Calculation

To map points in image coordinates to points in world coordinates, it is necessary to calculate the rotation (R) and translation (t) matrices of the camera according to its calibration model. This step was performed on an undistorted image. The code used for these operations is shown in Figure 8.4.

```
% Detect the checkerboard.
[imagePoints, boardSize] = detectCheckerboardPoints(im);

% Extract camera intrinsics.
camIntrinsics = cameraParams.Intrinsics;

% Adjust the imagePoints so that they are expressed in the coordinate system
% used in the original image, before it was undistorted.  This adjustment
% makes it compatible with the cameraParameters object computed for the
original image.
newOrigin = camIntrinsics.PrincipalPoint - newIntrinsics.PrincipalPoint;
imagePoints = imagePoints + newOrigin; % adds newOrigin to every row of
imagePoints

% Compute extrinsic parameters of the camera.
camExtrinsics = estimateExtrinsics(imagePoints, worldPoints, camIntrinsics);
```

Figure 8.4.Code for extracting internal and external parameters.

The resulting endogenous matrix K and exogenous matrices R (rotation matrix) and t (translation matrix) are shown in equations (8.1), (8.2) and (8.3);

$$K = \begin{bmatrix} 896.4296 & 0 & 0 \\ 0 & 895.3094 & 0 \\ 306.3769 & 255.0151 & 1 \end{bmatrix} \tag{8.1}$$

$$R = \begin{bmatrix} -0.0005 & -0.9995 & -0.0301 \\ 0.9994 & 0.0005 & -0.0340 \\ 0.0340 & -0.0301 & 0.9990 \end{bmatrix} \tag{8.2}$$

$$t = \begin{bmatrix} -188.4790 \\ 166.6233 \\ 991.3162 \end{bmatrix} \tag{8.3}$$

9. IMPLEMENTATION OF THE DELTA ROBOT SORTING SYSTEM

After establishing the motion control of the system application, the computer vision component of this system is examined. The main objective of this section is to integrate the results obtained in kinematic analysis (Chapter 5) with deep learning-supported object detection and image processing algorithms. A picking position calculation method is proposed to calculate the simultaneous positioning information of moving objects on the conveyor. This technique is used to provide updated positioning information of objects on the conveyor, thus determining the instantaneous position of the target object.

9.1. Homing and Camera Calibration

Once the process begins, the robot's motors are engaged, and the Delta robot moves to its home position. This is accomplished using an inductive sensor. Following this step, camera calibration is performed using the method described in the previous section.

Camera calibration was performed to calculate the intrinsic and extrinsic parameters. The main purpose of obtaining these parameters is to calculate the real-world coordinates of a point from the corresponding image coordinates. Figure 8.3 shows the projection error of the camera calibration process. The overall average error was found to be 0.11. This error condition is sufficient for this application. The resulting intrinsic matrix K and extrinsic matrices R (rotation matrix) and t (translation matrix) are shown in equations (8.1), (8.2) and (8.3).

9.2. Object Detection Process with Deep Learning

When the camera captures moving objects on the conveyor, this image is processed by the visual processing module of the computer vision system. Using a trained object detector, it identifies PET bottle-class objects among other objects in the waste line. Deep learning object detection is a fast and accurate way to classify objects in an image, and the necessary features for classification are learned automatically. A number of algorithms exist for deep learning-assisted object detection, such as Faster R-CNN, You Only Look Once (YOLO), and Single Shot Detection (SSD) (see Figure 9.1).

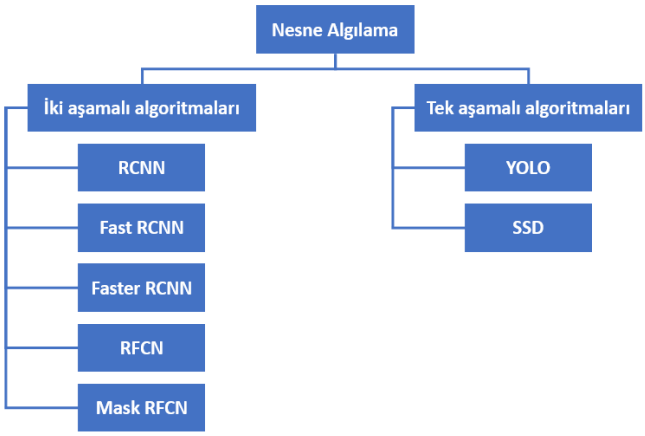


Figure 9.1.Single-stage and two-stage object detection algorithms.

In this study, two object detection models, SSD and YOLOv2, were selected. Both models use a single-stage object detection network, which makes them faster than two-stage models like Faster R-CNN (see Figure 9.2). The biggest difference between YOLOv2 and SSD lies in their approach to object detection (Pathak et al., 2018). YOLO divides the input image into a grid and estimates bounding boxes and class probabilities for each grid cell, while SSD creates multiple bounding boxes at different scales and aspect ratios for each feature map location. SSD and YOLOv2 consist of two subnets: a feature extraction network and a detection network. The former is a pre-trained CNN, with ResNet-50 used for feature extraction in this implementation. Secondly, it is a smaller CNN compared to the feature extraction subnet, consisting of several convolutional layers and layers specific to the deep learning model (Liu et al., 2016) (Liu et al., 2022).

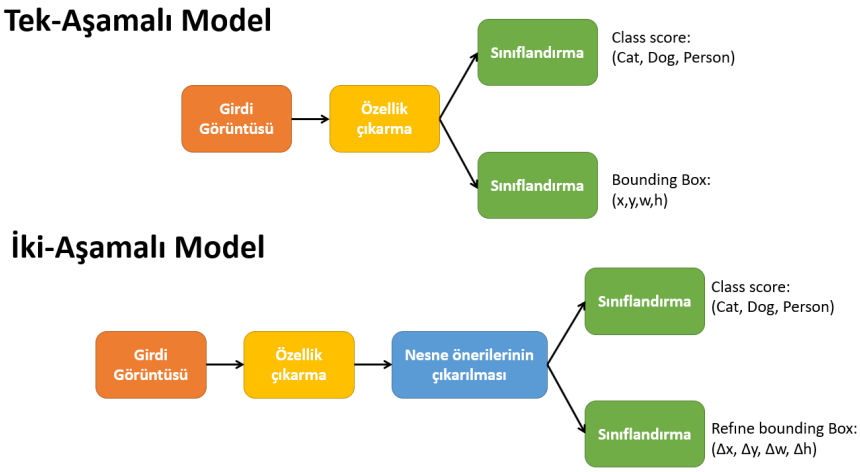


Figure 9.2.Architecture of single-stage and two-stage models

9.2.1. Data Set Collection, Adding and Expanding Information Notes

A dataset of 91 images was used to train the object detection network. The images comprising the dataset were captured using a Logitech C310 HD webcam. Several examples from the collected dataset are shown in Figure 9.3.

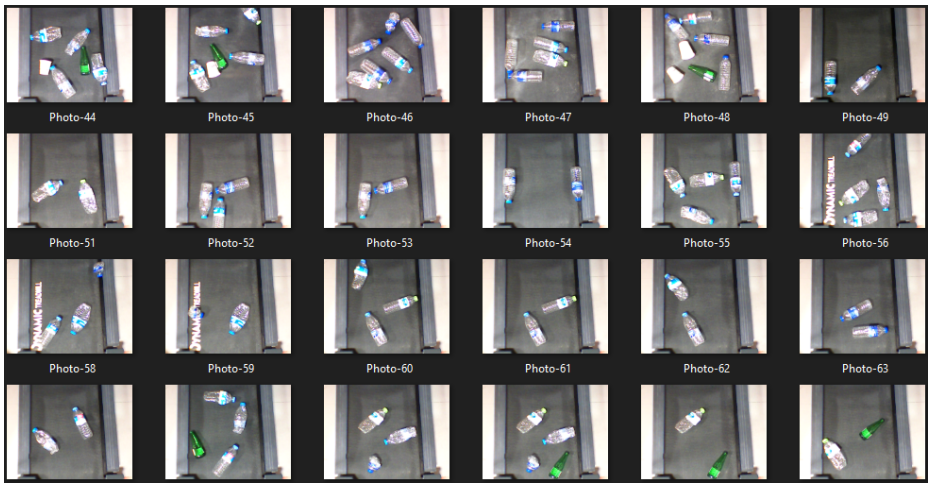


Figure 9.3.Examples from the collected dataset

After the images are collected, information is added and augmented using MATLAB image processing and deep learning toolboxes. Augmentations applied to the images include vertical flips and rotations. These simple augmentations were chosen due to their ease of implementation and computational efficiency. Using these augmentations increases the size of the dataset, improving generalization and preventing overfitting.

YOLOv2 and SSD models are mid-range models that were pre-trained on the COCO dataset and fine-tuned on the collected PET bottle dataset. Using these models provides a suitable balance between speed and accuracy.

9.2.2. Object Detection Model Training, Results and Evaluation

During the training process, these models were trained for 30 epochs, during which their weights were finely tuned to minimize the loss function. The training loss and root mean square error (RMSE) graphs obtained for the models are shown in Figures 9.4 and 9.5.

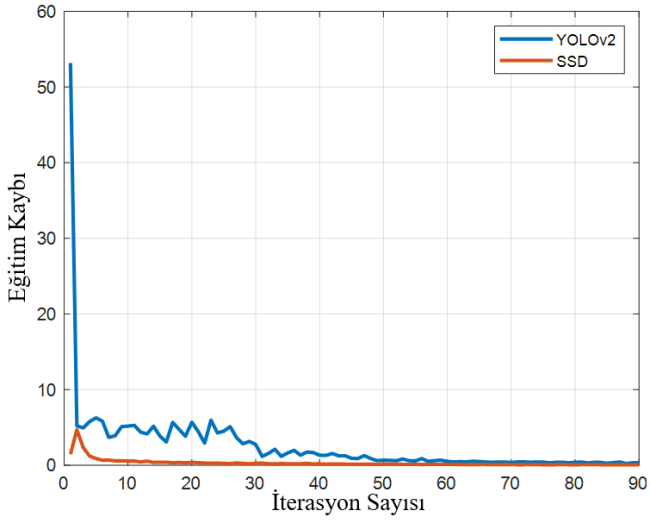


Figure 9.4.YOLOv2 and SSD Learning Loss graph

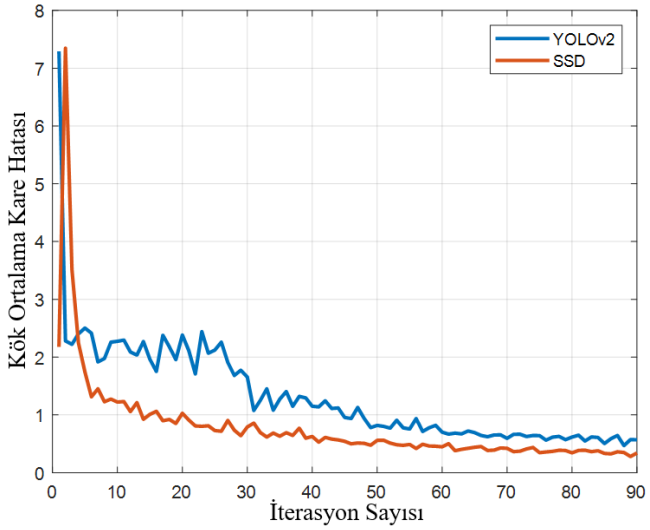


Figure 9.5.YOLOv2 and SSD Learning RMSE graph

To evaluate the performance of object detection models, namely SSD and YOLOv2, various metrics such as precision and recall were considered. The ideal precision is 1 at all recall levels. The average accuracy (AP) of the trained SSD and YOLOv2 models was found to be 0.7991 and 0.8742, respectively. An AP value of 0.8742 is a good value and reveals that the trained YOLOv2 model performs relatively well in terms of accuracy compared to SSD when the classification threshold changes. The accuracy/recall (PR) curve of the two models is shown in Figure 9.6. Experimental object detection results obtained from the YOLOv2 model are shown in Figure 9.7.

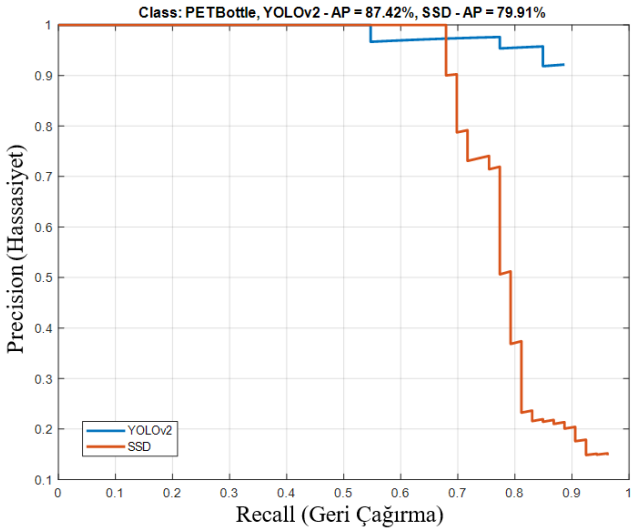


Figure 9.6.Accuracy/Recall rate graph of YOLOv2 and SSD.

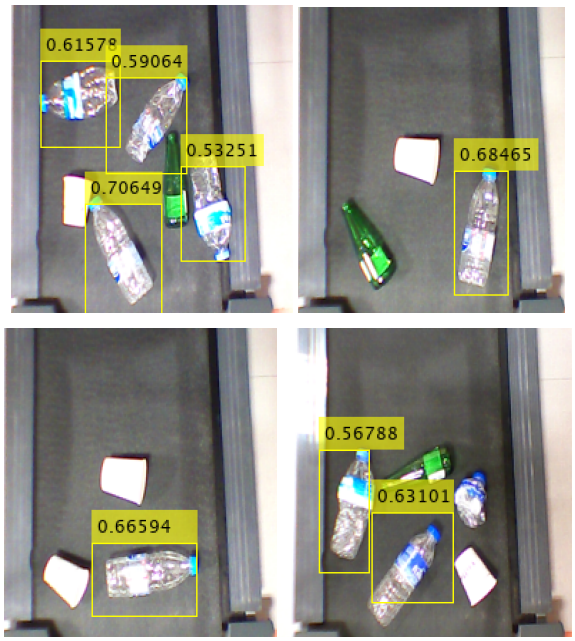


Figure 9.7.Results of the YOLOv2 model on test images.

9.3. Visual Positioning

Once the object detection process is complete, visual positioning calculations are performed to determine the object's position on the conveyor. The coordinates of the objects are obtained using the intrinsic and extrinsic parameters derived from the camera calibration process (section 8). The visual positioning results for an example case are shown in Figure 9.8.

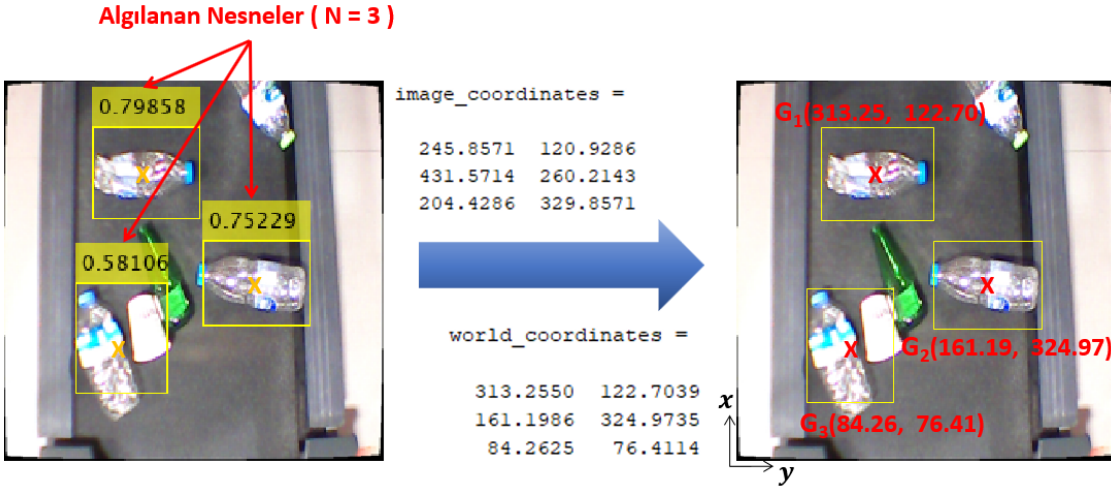


Figure 9.8.Visual positioning results

9.4. Collection Location Calculation Method

(x_n, y_n)It is the position of the next collection object point. Since it moves in the direction of the conveyor axis, its value remains constant as it changes. Therefore, the real-time position of the object is calculated using Equation (9.1);$P_n x x_n y_n$

$$\begin{cases} x_n(0)=x_0 \\ y_n(0)=y_0 \\ x_n(i+1)=x_n(i)+v.t(i) \\ y_n(i+1)=y_0 \end{cases}, n=1,2,\ldots,N \wedge i=0,1,\ldots,N \tag{9.1}$$

Here, the position coordinate of the target point at the moment the image is captured, the number of detected objects, the speed of the conveyor and the code execution time are represented by the formula expressed in Equation (9.2);$(x_0, y_0) N v t(i) t(i)$

$$t(i)=\begin{cases} t_c, \wedge\ i=0 \\ t_{pp}, \wedge\ i\geq 1 \end{cases} \tag{9.2}$$

Here, is the code execution time after image capture, and is the time required for each object acquisition and placement operation. Figure 9.9 shows the variation of code execution time for 50 different experimental trials. In this application, the time was set to constant. Thus, angular velocities for each arm of the delta robot were found using angle values calculated using the kinematic model.$t_c t_{pp} t_{pp}$

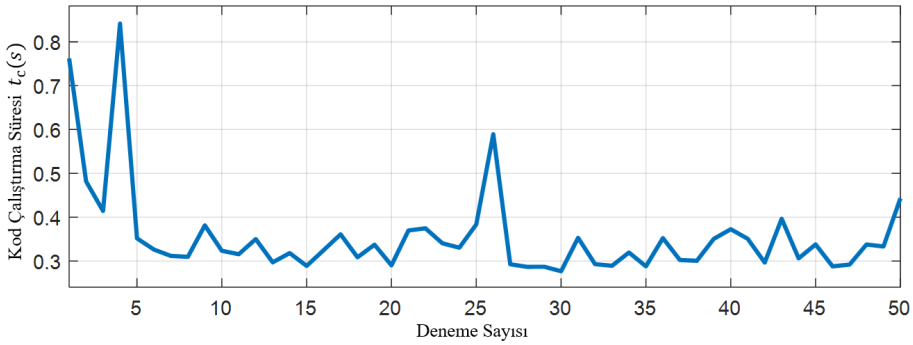


Figure 9.9.Code execution time for different trials

As shown in Figure 9.8, three objects were detected, namely . The coordinates of the calculated point targets are , , 'type. Therefore, the initial values of were calculated as follows;$N=3G_1(x_1(0),y_1(0))G_2(x_2(0),y_2(0))G_3(x_3(0),y_3(0))x_n(i)$

$x_1(0)=313.25$

$x_2(0)=161.19$

$x_3(0)=84.260$

According to the equation proposed above, we obtain the following:

If so:$i=0$

$x_1(1)=x_1(0)+v.t(0)=x_1(0)+v.t_c$

$x_2(1)=x_2(0)+v.t(0)=x_2(0)+v.t_c$

$x_3(1)=x_3(0)+v.t(0)=x_3(0)+v.t_c$

If so:$i=1$

$x_1(2)=x_1(1)+v.t(1)=x_1(1)+v.t_{pp}$

$x_2(2)=x_2(1)+v.t(1)=x_1(1)+v.t_{pp}$

$x_3(2)=x_3(1)+v.t(1)=x_1(1)+v.t_{pp}$

If so:$i=2$

$$x_1(3) = x_1(2) + v.t(2) = x_1(2) + v.t_{pp}$$

$$x_2(3) = x_2(2) + v.t(2) = x_1(2) + v.t_{pp}$$

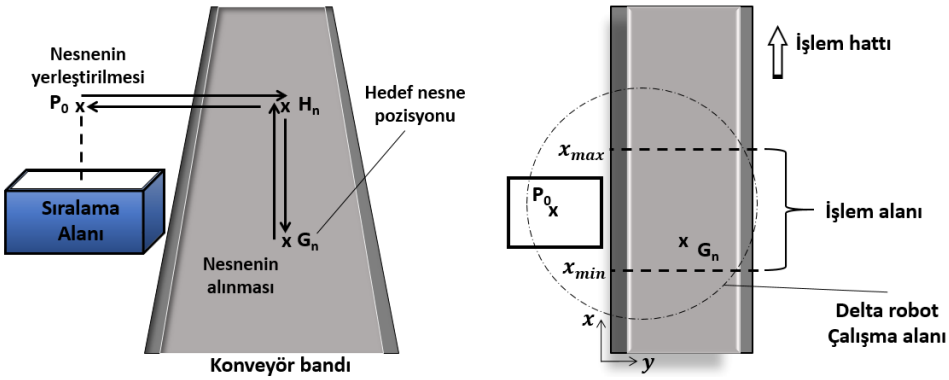
$$x_3(3) = x_3(2) + v.t(2) = x_1(2) + v.t_{pp}$$

For the case shown in Figure 9.8, the calculated values for and are given in detail in Table 9.1:$x_1 x_2 x_3$

Table 9.1., and calculated values for$x_1 x_2 x_3$

i	$x_1(i)$	$x_2(i)$	$x_3(i)$
0	313.25	161.19	84.26
1	**337.25**	185.19	108.26
2	-	**235.19**	158.26
3	-	-	**208.26**

Subsequently, the collection and placement paths are established from the starting point, as shown in Figure 9.10, to the next target point, which is considered the starting point. At this point, the collected object is released to fall into the sorting area box. The reason for passing before is to allow the delta end function to move above the target object position. This step is important for being able to pick up the object by pushing and pulling it perpendicularly from the conveyor. The push vector towards the object occurs in the direction of . After picking up the object, the pull from the object occurs in the direction of . The calculated coordinates for the waypoints of each detected object are detailed in Tables 9.2, 9.3, and 9.4.$P_0 H_n G_n P_0 H_n G_n \overrightarrow{H_n G_n} \overrightarrow{G_n H_n}$



Figures 9.10.The path created for the pick-and-drop operation.

After waypoints were established, the angular velocity values of the delta robot arms

were calculated for each step.$\omega_1\omega_2\omega_3$

Table 9.2.Data values for obtaining the first objective.

Point	x	y	z	θ_1	θ_2	θ_3	ω_1	ω_2	ω_3
P0	150	0	-1025	14.07	38.91	-13.2	0	0	0
H1	337.25	122.70	-1025	26.87	52.67	68.06	12.80	13.76	81.26
G1	**337.25**	122.70	-1100	9.87	35.67	51.06	17	17	17
H1	337.25	122.70	-1025	26.87	52.67	68.06	17	17	17
P0	150	0	-1025	14.07	38.91	-13.2	12.80	13.76	81.26

Table 9.3.Data values for achieving the second objective.

Point	x	y	z	θ_1	θ_2	θ_3	ω_1	ω_2	ω_3
P0	150	0	-1025	14.07	38.91	-13.2	0	0	0
H2	235.19	324.97	-1025	24.79	88.21	84.04	10.72	49.3	97.24
G2	**235.19**	324.97	-1100	7.79	71.21	67.04	17	17	17
H2	235.19	324.97	-1025	24.79	88.21	84.04	17	17	17
P0	150	0	-1025	14.07	38.91	-13.2	10.72	49.3	97.24

Table 9.4.Data values for achieving the third objective.

Point	x	y	z	θ_1	θ_2	θ_3	ω_1	ω_2	ω_3
P0	150	0	-1025	14.07	38.91	-13.2	0	0	0
H3	208.26	76.41	-1025	28.07	63.11	48.51	14.00	24.2	61.71
G3	**208.26**	76.41	-1100	11.07	46.11	31.51	17	17	17
H3	208.26	76.41	-1025	28.07	63.11	48.51	17	17	17
P0	150	0	-1025	14.07	38.91	-13.2	14.00	24.2	61.71

9.5. Communication between Robotics and Computer Vision Systems

To transmit the object's positioning information to the robotic control system, a communication interface has been established between the main systems. In this study, OPC over TCP/IP protocol was used (Liu et al., 2022). In the control architecture, as shown in Figure 9.11, the computer vision system is used as the server and the robotic control system as the client. To ensure successful communication between the DPR and the computer vision system, the transmitted signals must follow the same communication protocol.

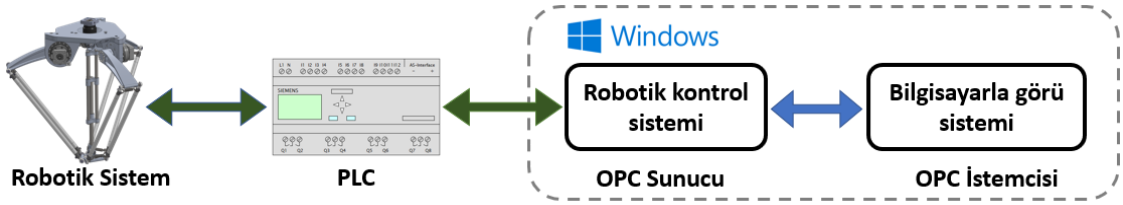


Figure 9.11.Computer vision and motion control systems communication interface

9.6. Integration of Computer Vision System with Delta Robot

In this study, a computer vision system was integrated with a delta robot to separate PET bottles in a waste pipeline.

The delta's gripper is created with a vacuum suction nozzle. Initially, the delta is set to its starting point. When the flow path begins, the computer vision system starts processing. When the object is fully visible on the screen, the visual processing module detects the PET bottles and determines the number of detected objects. Then, the visual positioning module performs positional analysis to calculate target points and create collection paths. Subsequently, the positional information of the target points is updated following the suggested method. The necessary information is transmitted to the robotic control system via communication networks. After receiving the information, the PLC performs the operation by collecting the detected PET bottles from other objects and placing them in the sorting area.

The pseudocode for the working system is given in Algorithm 1. Figure 9.12 also shows the steps of the parsing process.

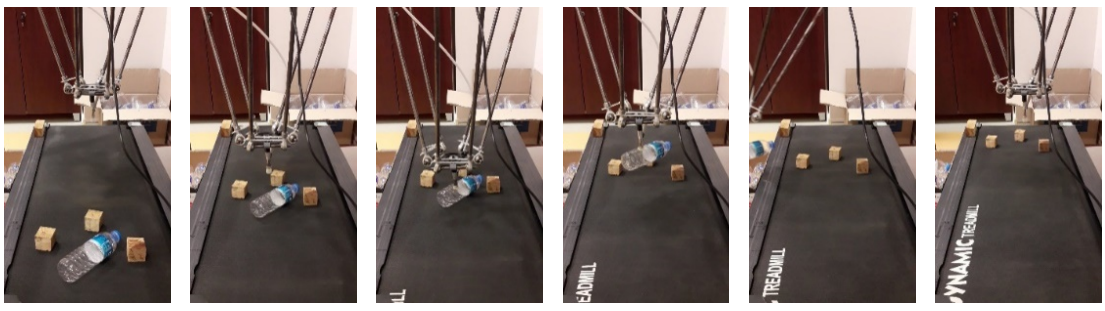


Figure 9.12.The object parsing sequence is: detecting the object, approaching the object, picking up the object, and dropping the object into the parsing area.

Algorithm: The pseudocode of the system.

```
Start
Starting the System
Do not homing
Camera Calibration
If the while loop termination criterion is not met, do
    Take a picture
    Image processing
    Object detection
    Visual positioning
    K←Number of target points

    if K == 0
        continue
    end if

    for i←1 to K do
        Plan the route points.
        Estimate location
        Cālculate angle values.
        Get information about the target locations.
        Motion_control()
    end for
end while

function Motion_control()
Read the information for the next destinations.

if the target point is not within the working area
    Go to step 23.
end if

Calculate speed values.
Calculate robot positioning.
Write to the PLC's data block.
end function
```

Figure 9.13. Pseudocode of the implementing system.

10. RESULTS AND DISCUSSION

This master's thesis examines the feasibility of using deep learning for computer vision systems in a robotic system (Delta parallel robot) for industrial automation. The literature review provides an overview of applications requiring deep learning algorithms, along with examples of where and how computer vision systems can be implemented. Two broad areas where deep learning for computer vision is used in industrial applications were identified: object detection and visual positioning.

Coordinates from outside the working area were not considered. 100% success was achieved in reconstructing the coordinates extracted from the images using kinematic analysis, and data for each trajectory of the delta robot arm was successfully transferred from the OPC Server to the OPC data in the PLC Data Block.

One of the important observations in this study is that some mechanical errors affecting the accuracy of the movement were encountered in obtaining different z-position results for some situations. This situation is thought to be due to the robot's own design. Therefore, parallel delta robots are preferred over linear delta robots for pick-and-drop operations.

Experiments have shown that computer vision-assisted position measurements can be used and utilized as feedback for a robotic application. However, it should be noted that all experiments were conducted in a laboratory setting under stable conditions, and other problems may arise in a real-world industrial application. Potential problems include varying lighting conditions, dirty environments, and/or the display of irregularly shaped objects.

An examination of the results obtained from the pick-and-drop experiment revealed that previous studies (as discussed in the literature review) showed that encoder-based inferences performed slightly better than computer vision-based inference in terms of error distribution for both speed and position. Although encoder-based inference showed better performance, the results indicated that computer vision-based inference is feasible in this type of pick-and-drop application, given that the application is within the accuracy level of the computer vision system. It should be noted that the working system was not optimized for higher accuracy, meaning that it is possible to achieve

even higher resolution than presented. This can be achieved by applying modifications based on factors that have been identified as affecting accuracy, such as higher camera resolution.

Another notable observation is that the delta robot captures the object either in front of or behind its center point. This is thought to be a calibration error that occurs when translation and rotation measurements are taken to convert the computer vision system's coordinate system to the delta robot's coordinate system. One way to eliminate this is to use more sophisticated methods for calibration.

Finally, this study demonstrates that the computer vision-assisted robotic system studied can be used to separate plastic PET bottles from a waste line. The separation process is performed by carrying out pick-and-place tasks for a waste line moving on a conveyor belt. A computer vision system consisting of modules was written in MATLAB to perform image processing and deep learning-based object detection operations. The You Only Look Once (YOLO) method was studied for object detection operations. Then, motion planning and position calculation methods were used to calculate target positions using the proposed approach. Finally, the visual processing results, position analysis, and motion control were integrated to automatically sort the detected objects. The evaluation results support the fact that the proposed system is both applicable and effective for PET bottle separation purposes. Furthermore, it shows a high correlation between the performance of the delta parallel robot and the object detection technique used, indicating that the two work well together. It is important to consider that the effectiveness of the studied system is limited by some mechanical, calibration, and detection errors such as the hardware used, the physical complexity of the objects, the lighting of the working environment, and the quality of the camera.

This study focuses primarily on the literature review, experiments, and applications carried out in a controlled laboratory environment. Therefore, examining in more detail how real-time computer vision applications are used in practice in the industry today, for example by interviewing industry experts, seems beneficial in gaining a more comprehensive perspective on the subject. In addition, seeing the results drawn from the experiments and how the developed model is translated into a real-world industrial application is also an important point.

In the experiment, only one method was used to calculate the arm speed of the delta robots, namely taking into account the time and position difference of the object between two frames. It is thought that it would be better to compare the method used with the optical flow method, where the speed is determined based on the perceived movement of individual pixels.

If the application used in this study is to be used in an industrial environment, it can be further improved to create a more reliable and higher-performance system. This solution should then be optimized based on the results obtained from experiments, for example, by using a higher-resolution camera.

SOURCES

Alvarez, H.H., Bolivar, E., Cervantes, C., Cuadros, E.E., Barrios, D., Patiño, R. (2014). Automation of chestnuts selection process using computer vision in real time. 33rd International Conference of the Chilean Computer Science Society (SCCC), Talca, Chile, 2014, 87-91. DOI: 10.1109/SCCC.2014.21.

Arreguin, J. M. R. (2008). Automation and Robotics. I-Tech Education and Publishing, Vienna. DOI: 10.5772/91

Bonev, I. (2001). The Delta parallel robot-the story of success.http://www.parallemic.org/Reviews/Review002.html(Date of visit: December 28, 2023).

Bouguet, J. Y. (2003). Camera Calibration Toolbox for Matlab. http://robots.stanford.edu/cs223b04/JeanYvesCalib/ (Date of Visit: April 19, 2024).

Bingül, Z., Küçük, S. (2005). Robotics, Birsen Publishing House, Istanbul, 337 pages.

Clavel, R. (1991). Conception d'un robot parallèle rapide à 4 degrés de liberté. Doctoral Thesis, Ecole Polytechnique Federale de Lausanne, Lausanne, Switzerland, 925.

Chen, Y. W., Shiu, J. M. (2022). An implementation of YOLO-family algorithms in classifying the product quality for the acrylonitrile butadiene styrene metallization. Int J Adv Manuf Technol 119, 8257–8269. DOI: 10.1007/s00170-022-08676-5

Cong, V.D., Phuong, L.H. (2023). Design and development of a delta robot system to classify objects using image processing. International Journal of Electrical and Computer Engineering (IJECE), 2669-2676. DOI: 10.11591/ijece.v13i3.pp2669-2676

Courtney, J., Magee, M., Aggarwal, J. (1984). Robot Guidance Using Computer Vision. Pattern Recognition, 585–592. DOI: 10.1016/0031- 3203(84)90012-8.

Doğan, E. (2010). Forward and Inverse Kinematics Calculations of Delta Parallel Robots. Master's Thesis, Eskişehir Osmangazi University, Odunpazarı, Eskişehir, 252664.

Erdoğan, K., Yılmaz, N. (2014). Shifting Colors to Overcome not Realizing Objects Problem due to Color Vision Deficiency. 2nd Int. Conf. on Advances in Computing, Electronics and Electrical Technology - CEET, 11–14. DOI: 10.15224/978-1-63248-034-7-27

Farag, M., Ghafar, ANA, Alsibai, M.H. (2019). Real-time robotic grasping and localization using deep learning-based object detection technique. IEEE International Conference on Automatic Control and Intelligent Systems

(I2CACIS), Selangor, Malaysia. 139–144. DOI: 10. 1109/I2CACIS.2019.8825093.

Foley, D., O'Reilly, R. (2018). An evaluation of convolutional neural network models for object detection in images on low-end devices, in AICS, 2018, vol. 2259 (2018), pp. 1–12

Gasparetto, A., Scalera, L. (2019). From the Unimate to the Delta Robot: The Early Decades of Industrial Robotics. In: Zhang, B., Ceccarelli, M. (eds) Explorations in the History and Heritage of Machines and Mechanisms. History of Mechanism and Machine Science, vol 37. DOI: 1007/978-3-030-03538-9_23

Guglielmetti, P. (1994). Model-based control of fast parallel robots: a global approach in operational space. Doctoral Thesis, Ecole Polytechnique Federale de Lausanne, Lausanne, Switzerland, 1228.

Gholami, A., Homayouni, T., Ehsani, R., Sun, J. (2021). Inverse Kinematic Control of a Delta Robot Using Neural Networks in Real-Time. Robotics. 10(4):115. DOI: 10.3390/robotics10040115

Hagara, M., Stojanovic, R., Bagala, T., Kubinec, P., Ondrácek, O. (2020). Grayscale image formats for edge detection and for its FPGA implementation. Microprocessors and Microsystems, 103056. DOI: 10.1016/j.micpro.2020.103056.

Heikkila, J., Silven, O. (1997). A Four-step Camera Calibration Procedure with Implicit Image Correction. IEEE International Conference on Computer Vision and Pattern Recognition. 1106-1112. DOI: 10.1109/CVPR.1997.609468.

Joblove, G.H., Greenberg, D. (1978). Color spaces for computer graphics. Proceedings of the 5th annual conference on Computer graphics and interactive techniques. 20–25. DOI: 10.1145/ 965139.807362.

Lies, BT, Cai, Y., Saphr, E., Lin, K., Qin, H. (2018). Machine vision assisted micro-filament detection for real-time monitoring of electrohydrodynamic inkjet printing. Procedia Manufacturing 26. 46th SME North American Manufacturing Research Conference, NAMRC 46, Texas, USA. 29–39. DOI: 10.1016/j.promfg.2018.07.004.

Lin C.J., Shaw, J., Tsou, P.C., Liu, C.C. (2016). Vision servo-based Delta robot to pick-and-place moving parts. 2016 IEEE International Conference on Industrial Technology (ICIT), Taipei, Taiwan. 1626-1631. DOI: 10.1109/ICIT.2016.7475005

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, CY, Berg, AC (2016). SSD: Single Shot MultiBox Detector. In: Leibe, B., Matas, J., Sebe, N., Welling, M. (eds). Computer Vision – ECCV 2016.21-37. DOI:10.1007/978-3-319-46448-0_2

Liu, W., Wan, P., Cheng, J., Ma, Y., Jing, C. (2022). High-precision position control of belt drive system based on OPC communication. Int J Adv Manuf Technol, 1–10. DOI: 10.1007/s00170-021-07859-w

Liu, Y., Hu, T., Ni, H., Zhang, C., Lin, B., Judd, E. (2018). Design of a pc-based open industrial robot control system integrated with real-time machine vision. Int: 2018 WRC Symposium on Advanced Robotics and Automation (WRC SARA), Beijing, China. 1–6. DOI: 10.1109/WRC-SARA.2018. 8584214.

LaValle, S. M. (2006). Planning Algorithms. Cambridge University Press, Cambridge, UK DOI: 10.1017/CBO9780511546877.

Nzue, RMA, Brethé, J.F., Vasselin, E., Lefebvre, D. (2013). Comparison of Serial and Parallel Robot Repeatability Based on Different Performance Criteria. Mechanism and Machine Theory. 136-155. DOI:10.1016/j.mechmachtheory.2012.10.004

Pathak, A., Pandey, R., Rautaray, MS (2018). Application of Deep Learning for Object Detection, Procedia Computer Science. 1706-1717, DOI: 10.1016/j.procs.2018.05.144

Peršak, T., Viltužnik, B., Hernavs, J., Klancnik S. (2020). Vision-based sorting systems for transparent plastic granulate. Applied Sciences. 4269. DOI: 10.3390/app10124269.

Qingmao, H., Hou, Z., Nowinski, W. L. (2006). Supervised range-constrained thresholding. IEEE Transactions on Image Processing. 228–240. DOI:10.1109/TIP.2005.860348.

Schmitt, R., Fürtjes, T., Abbas, B., Abel, P., Kimmelmann, W., Kosse, P., Buratti, A. (2015). Real-time machine vision system for an automated quality monitoring in mass production of multiaxial non-crimp fabrics. 15th IFAC Symposium on Information Control Problems in Manufacturing, Ottawa, Canada. 2393–2398. DOI: 10.1016/j.ifacol.2015.06.446.

Smirnova, Y. (2023). Delta-robot kinematic, MATLAB Central File Exchange. https://www.mathworks.com/matlabcentral/fileexchange/71119-delta-robot-kinematic, (Accessed: December 9, 2023)

Solovyev, R., Wang, W., Gabruseva, T. (2021). Weighted boxes fusion: ensembling boxes from different object detection models. Image and Vision Computing. 104117. DOI: 10.1016/j.imavis.2021.

Tiwari, S. (2016). An introduction to qr code technology. International Conference on Information Technology (ICIT), Bhubaneswar, India. 39–44. DOI: 10.1109/ICIT.2016.021.

URL-1: https://www.analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-for-

camera-calibration-in-computer-vision/ (Accessed: December 22, 2024)

URL-2:https://de.mathworks.com/help/vision/ug/camera-calibration.html(Date of visit: December 22, 2024)

URL-3:https://www.mathworks.com/help/vision/ug/fisheye-calibration-basics.html(Date of visit: December 22, 2024)

URL-4:https://learnopencv.com/understanding-lens-distortion/(Date of visit: December 22, 2024)

URL-5:https://www.foamcoreprint.com/blog/what-are-calibration-targets(Date of visit: December 22, 2024)

URL-6:https://www.analyticsvidhya.com/blog/2022/03/a-basic-introduction-to-object-detection/(Date of visit: December 22, 2024)

URL-7:https://levity.ai/blog/difference-machine-learning-deep-learning(Date of visit: December 22, 2024)

URL-8:https://nrsyed.com/2017/12/17/animating-the-jacobian-inverse-method-with-an-interactive-matplotlib-plot/(Date of visit: December 22, 2024)

URL-9: Online Documentation, Access date: August 17, 2024,https://delta-ia-tips.com/2012/11/05/technical-announcement-pnp-connection-to-delta-servodrive/(Date of visit: December 22, 2024)

URL-10: Online Documentation, Access date: August 19, 2024,https://support.industry.siemens.com/cs/mdm/107623221?c=73837491339&lc=en-AE(Date of visit: December 22, 2024)

Williams II, R. L. (2016). The Delta Parallel Robot: Kinematics Solutions.www.ohio.edu/people/williar4/html/pdf/DeltaKin.pdf. (Accessed: January 26, 2024)

Yu, LS, Chou, SY, Wu, HY, Chen, YC, Chen, YH (2020). Rapid and semi-quantitative colorimetric loop-mediated isothermal amplification detection of asfv via hsv color model transformation. Journal of Microbiology, Immunology and Infection. 963-970. DOI: 10.1016/j.jmii.2020.08.003.

Zaera, M., Esteve, M., Palau, CE, Guerri, JC, Martinez, FJ, Decordoba, PF (2001). Real-time scheduling and guidance of mobile robots on factory floors using Monte Carlo methods under windows. 8th International Conference on Emerging Technologies and Factory Automation. Antibes-Juan les Pins, France. 67-74. DOI: 10.1109/ETFA.2001.996355.

Zhao, J., Xia, X., Wang, H., Kong, S. (2016). Design of real-time steel bars recognition system based on machine vision. 2016 8th International Conference on Intelligent Human-Machine Systems and Cybernetics (IHMSC), Hangzhou, China. 505–509.

DOI: 10.1109 / IHMSC.2016.75.

Zhang, Z. (2000). A Flexible New Technique for Camera Calibration. IEEE Transactions on Pattern Analysis and Machine Intelligence. 1330–1334. DOI: 10.1109/34.888718.

PERSONAL PUBLICATIONS AND WORKS

Chachoua, ME, Öztürk, S. (2024). Application of PLC Controlled Delta Robot PET Bottle Sorting System with Deep Learning. In F. Kuncan, A. Yıldırım (Eds.), Interdisciplinary Academic Studies (2nd ed.), İKSAD, Ankara, Turkey. (61-93). DOI: 10.5281/zenodo.13825221

RESUME

He completed his primary, middle, and high school education in Algeria. In 2017, he began his four-year studies in Electronic and Communication Engineering at Kocaeli University, graduating in 2021. He started his master's program in Electronic and Communication Engineering at the Kocaeli University Institute of Science in 2022, graduating in the fall semester of 2024.